INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO
CAMPUS SÃO PAULO
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

AUGUSTO HENRIQUE BRAGA SILVA
HENRIQUE ORSONI
LUCIANO RICARDO PAIM DE ARAUJO
RENATO VANIL DE OLIVEIRA
VITORIA DE LIMA SANTOS

CREATOR WORKBENCH

SÃO PAULO, SÃO PAULO
2026

AUGUSTO HENRIQUE BRAGA SILVA
HENRIQUE ORSONI
LUCIANO RICARDO PAIM DE ARAUJO
RENATO VANIL DE OLIVEIRA
VITORIA DE LIMA SANTOS

CREATOR WORKBENCH

Trabalho da disciplina PROJETO INTEGRADO DE EXTENSÃO 1 apresentado como exigência parcial para obtenção do diploma do Curso de Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, Campus São Paulo.

Orientador: Prof. M^{e}. Marcelo Tavares de Santana e Johnata Souza Santicioli

SÃO PAULO, SÃO PAULO
2026

AUGUSTO HENRIQUE BRAGA SILVA
HENRIQUE ORSONI
LUCIANO RICARDO PAIM DE ARAUJO
RENATO VANIL DE OLIVEIRA
VITORIA DE LIMA SANTOS

CREATOR WORKBENCH

Trabalho da disciplina PROJETO INTEGRADO DE EXTENSÃO 1 apresentado como exigência parcial para obtenção do diploma do Curso de Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, Campus São Paulo.

Aprovada em _____/_____/_____.

BANCA EXAMINADORA:

Prof. M^{e}. Marcelo Tavares de Santana e Johnata Souza Santicioli(Orientador)
Instituto Federal de São Paulo (IFSP)

Prof. M^{e}. Johnata Souza Santicioli
Instituto Federal de São Paulo (IFSP)

SÃO PAULO, SÃO PAULO
2026

Dedico este trabalho aos nossos familiares, amigos e professores, que nos apoiaram e incentivaram durante toda a trajetória acadêmica.

Também dedicamos este projeto a todos que acreditam no poder da tecnologia, da educação e da inovação como ferramentas de transformação social.

AGRADECIMENTOS

Agradecemos ao Prof. Me. Marcelo Tavares de Santana e ao Prof. Me. Johnata Souza Santicioli, nossos orientadores, pela disponibilidade, direcionamento técnico e pela confiança depositada na equipe ao longo de todo o desenvolvimento deste projeto.

Aos professores do curso de Tecnologia em Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, Campus São Paulo, que ao longo da graduação nos forneceram os fundamentos técnicos e acadêmicos necessários para a realização deste trabalho.

Aos colegas de curso, pelo companheirismo e pelas trocas de conhecimento que enriqueceram nossa formação.

Por fim, agradecemos às nossas famílias e amigos, cujo apoio incondicional foi essencial para que chegássemos até aqui.

“A tecnologia move o mundo.” Steve Jobs

RESUMO

O crescimento acelerado da *Creator Economy* e a consolidação do ensino à distância no Brasil criaram uma demanda por ferramentas de gestão de conteúdo educacional que ferramentas genéricas como Trello, Notion e Asana não são capazes de suprir, por carecerem de estrutura semântica nativa para o domínio educacional. O presente trabalho descreve o desenvolvimento do Creator Workbench, uma plataforma *web* de *Content Operations* (ContentOps) projetada especificamente para centralizar e otimizar o *pipeline* de produção de criadores de conteúdo educacional digital. A metodologia adotada é o Scrum, com *Sprints* quinzenais, equipe de cinco integrantes com papéis técnicos distintos e repositório versionado no GitHub. A arquitetura do sistema segue o padrão *Client-Server* desacoplado, com *front-end* em React.js, *back-end* em Java com Spring Boot, banco de dados relacional MySQL normalizado até a Terceira Forma Normal e infraestrutura provisionada na Amazon Web Services. A plataforma oferece módulos de gestão de ideias, *workflow* Kanban com estados de produção, editor de roteiros, *upload* de ativos digitais integrado ao AWS S3, automações via *scripts* Python e estruturação curricular em Cursos, Módulos e Aulas. A segurança é implementada por autenticação JWT, criptografia BCrypt e comunicação HTTPS, em conformidade com a Lei Geral de Proteção de Dados Pessoais. A análise de viabilidade financeira demonstra sustentabilidade do modelo SaaS com três planos de assinatura, apresentando margem operacional superior a 90% a partir do sétimo mês no cenário realista. Os resultados indicam que uma ferramenta com semântica nativa para o domínio educacional reduz a fragmentação operacional do criador e cria condições técnicas para a aplicação dos princípios andragógicos na produção de cursos digitais.

Palavras-chave: *Creator Economy*. *Content Operations*. Gestão de Conteúdo Educacional. Educação à Distância. SaaS.

ABSTRACT

The accelerated growth of the Creator Economy and the consolidation of distance learning in Brazil have created a demand for educational content management tools that generic platforms such as Trello, Notion and Asana are unable to meet, as they lack native semantic structure for the educational domain. This paper describes the development of Creator Workbench, a web-based Content Operations (ContentOps) platform designed specifically to centralize and optimize the production pipeline for digital educational content creators. The methodology adopted is Scrum, with biweekly Sprints, a five-member team with distinct technical roles and a versioned repository on GitHub. The system architecture follows the decoupled Client-Server pattern, with a React.js front-end, a Java Spring Boot back-end, a MySQL relational database normalized to Third Normal Form and infrastructure provisioned on Amazon Web Services. The platform offers modules for idea management, a Kanban workflow with production states, a script editor, digital asset upload integrated with AWS S3, Python-based automations and curricular structuring into Courses, Modules and Lessons. Security is implemented through JWT authentication, BCrypt encryption and HTTPS communication, in compliance with the Brazilian General Data Protection Law (LGPD). The financial viability analysis demonstrates the sustainability of the SaaS model with three subscription plans, presenting an operating margin above 90% from the seventh month in the realistic scenario. The results indicate that a tool with native semantics for the educational domain reduces the creator's operational fragmentation and creates the technical conditions for applying andragogical principles in the production of digital courses.

Keywords: Creator Economy. Content Operations. Educational Content Management. Distance Learning. SaaS.

LISTA DE ILUSTRAÇÕES

Figura 1 – Interface da plataforma Trello 18
Figura 2 – Interface da plataforma Notion 19
Figura 3 – Interface da plataforma Asana 19
Figura 4 – Diagrama da Arquitetura Client-Server Desacoplado 35
Figura 5 – Diagrama de Componentes (*White-Box*) - Creator Workbench . . . 37
Figura 6 – Diagrama de Implantação - Creator Workbench 38
Figura 7 – Modelo Entidade-Relacionamento - Creator Workbench 44
Figura 8 – Diagrama Entidade-Relacionamento - Creator Workbench 45
Figura 9 – Diagrama de Classes - Creator Workbench 47
Figura 10 – Tela de Autenticação – Acesso restrito com campos de credencial e código de acesso 58
Figura 11 – *Dashboard* Principal – Caixa de Entrada com *cards* de ideias e contadores de *Ideias_Insight* e Status_Produção 58
Figura 12 – Menu Lateral – Navegação entre Dashboard Geral, Ideação e Configurações, com opções de personalização e encerramento de sessão 59
Figura 13 – *Modal* Capturar *Insight* – Formulário de cadastro rápido de ideia com campos de título e descrição 59
Figura 14 – Estúdio de Escrita – Editor de texto rico com suporte a formatação, revisões e snapshot de versões 60
Figura 15 – Projetos Ativos – *Pipeline_Ativo* com projeto em produção e painel de Estrutura Curricular com módulos e aulas planejadas 60

LISTA DE ABREVIATURAS E SIGLAS

3FN	Terceira Forma Normal
ABNT	Associação Brasileira de Normas Técnicas
ADS	Análise e Desenvolvimento de Sistemas
API	Application Programming Interface
AWS	Amazon Web Services
BCrypt	Blowfish Crypt, algoritmo de criptografia de senhas
CI	Continuous Integration
CMS	Content Management System
ContentOps	Content Operations
CSV	Comma-Separated Values
CT	Cenário de Teste
DBA	Database Administrator
DER	Diagrama Entidade-Relacionamento
DevOps	Development and Operations
EBS	Elastic Block Store
EC2	Elastic Compute Cloud
ECR	Elastic Container Registry
EKS	Elastic Kubernetes Service
ELB	Elastic Load Balancer
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IFSP	Instituto Federal de Educação, Ciência e Tecnologia de São Paulo
JPA	Java Persistence API
JSON	JavaScript Object Notation

JWT	JSON Web Token
LGPD	Lei Geral de Proteção de Dados Pessoais
LMS	Learning Management System
MER	Modelo Entidade-Relacionamento
MVP	Minimum Viable Product
NoSQL	Not only SQL
PDF	Portable Document Format
PO	Product Owner
RBAC	Role-Based Access Control
RDS	Relational Database Service
REST	Representational State Transfer
RF	Requisito Funcional
RN	Regra de Negócio
RNF	Requisito Não Funcional
S3	Simple Storage Service
SaaS	Software as a Service
SEO	Search Engine Optimization
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
UI	User Interface
URL	Uniform Resource Locator
US	História de Usuário
UUID	Universally Unique Identifier
UX	User Experience
VPC	Virtual Private Cloud

LISTA DE SÍMBOLOS

%	Porcentagem
R$	Real, moeda brasileira
US$	Dólar americano
_	Sublinhado
$\rightarrow$	Seta, indica a ordem da hierarquia (por exemplo, Curso $\rightarrow$ Módulo $\rightarrow$ Aula)

SUMÁRIO

1	**INTRODUÇÃO**	**15**
1.1	Objetivos	15
1.1.1	Objetivo Geral	16
1.1.2	Objetivos Específicos	16
1.2	Problema e Solução Proposta	16
1.3	Justificativa	17
1.4	Análise da Concorrência	17
1.4.1	Critérios de seleção	17
1.4.2	Trello	18
1.4.3	Notion	18
1.4.4	Asana	19
1.4.5	Comparativo	20
2	**FUNDAMENTAÇÃO TEÓRICA**	**21**
2.1	Histórico do Assunto	21
2.2	Atualidade do Assunto	21
2.3	Outros Contextos do Assunto	22
2.3.1	Andragogia	22
2.3.2	Arquitetura da Informação	23
3	**GESTÃO DO PROJETO**	**24**
3.1	Organização da Equipe	24
3.1.1	Responsabilidades, Papéis e Atividades	24
3.2	Metodologias de Gestão e Desenvolvimento	24
3.2.1	Scrum	25
3.2.1.1	*Sprints*	25
3.3	Repositório da Aplicação	26
3.3.1	Definição do Repositório	26
3.3.2	Link do Repositório	26
4	**DESENVOLVIMENTO DO PROJETO**	**27**
4.1	Escopo do Projeto	27
4.1.1	Regras de Negócio (RN)	27
4.1.2	Requisitos Funcionais (RF)	28
4.1.3	Requisitos Não Funcionais (RNF)	30
4.2	Histórias de Usuário	31

4.2.1 Descrição das Histórias de Usuário 31
4.2.2 Critérios de Aceitação em Gherkin 32
4.2.3 Rastreabilidade entre Histórias de Usuário e Testes 34
4.3 Arquitetura 34
4.3.1 Definições da Arquitetura 34
4.3.2 Diagrama da Arquitetura 35
4.3.3 Diagrama de Componentes 35
4.3.4 Diagrama de Implantação 36
4.3.5 Integração com o Serviço de Automação 36
4.4 Tecnologias 39
4.4.1 *Front-end* 39
4.4.2 *Back-end* 40
4.4.3 Banco de Dados 40
4.4.4 Infraestrutura 40
4.4.4.1 Amazon Web Services (AWS) 40
4.4.4.2 Docker e Contêineres 41
4.4.4.3 Integração Contínua (CI) 41
4.5 Testes e Manutenibilidade 41
4.5.1 Plano de Testes 41
4.5.2 Análise Estática 41
4.5.3 Cobertura e Resultados dos Testes 42
4.6 Segurança, Privacidade e Legislação 42
4.6.1 Critérios de Segurança e Privacidade 42
4.6.2 Observância à Legislação 42
4.7 Modelo de Banco de Dados 43
4.7.1 Modelo Entidade-Relacionamento (MER) 43
4.7.2 Figura do Modelo Entidade-Relacionamento 43
4.7.3 Diagrama Entidade-Relacionamento (DER) 44
4.7.4 Diagrama de Classes 45
4.7.5 Dicionário de Dados 46
4.8 Duração / Cronograma 48
4.8.1 Análise da Duração do Projeto 48

5 VIABILIDADE FINANCEIRA 50
5.1 Custos 50
5.2 Receitas 50
5.2.1 Cenário Realista 51
5.2.2 Cenário Otimista 52
5.2.3 Cenário Pessimista 52

6 CONSIDERAÇÕES FINAIS 53
6.1 Dificuldades, Escolhas e Descartes 53
6.2 Trabalhos Futuros 54

REFERÊNCIAS 55

APÊNDICES 57

APÊNDICE A – TELAS DA APLICAÇÃO – CREATOR WORKBENCH 58

1 INTRODUÇÃO

O avanço dos modelos híbridos e digitais na educação fez do conteúdo educacional um bem de grande valor. Hoje, a produção desse material não é mais exclusiva de grandes instituições e passou para as mãos de criadores independentes e pequenas empresas, a chamada *Creator Economy*. O Creator Workbench surge como uma ferramenta de *Content Operations* (*ContentOps*) para preencher o espaço entre a criatividade e a organização do trabalho necessária para produzir cursos e outros produtos educacionais.

Esse movimento tem origem em mudanças no comportamento de quem consome educação. A pandemia de COVID-19 acelerou o uso do ensino à distância, e a procura por cursos digitais passou a ser uma necessidade permanente, e não apenas uma solução de emergência. Segundo o Censo da Educação Superior 2024, as matrículas em cursos a distância já representam 50,7% do total de matrículas de graduação no Brasil, com crescimento de 5,6% entre 2023 e 2024 (Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira, 2025), o que mostra uma mudança profunda na forma como o conhecimento é consumido e vendido.

Na prática, transformar conhecimento em um curso organizado exige muito mais do que dominar o assunto. O criador de cursos digitais faz, ao mesmo tempo, o papel de roteirista, produtor de vídeo, gestor de projetos e estrategista de conteúdo. Com tantos papéis, organizar o trabalho se torna um grande desafio. Um curso tem uma ordem natural: as aulas dependem de roteiros, os roteiros dependem de pesquisas e os conjuntos de aulas formam os módulos. Sem um sistema que siga essa ordem, a produção se espalha entre aplicativos de notas, planilhas e pastas de arquivos sem padrão, o que causa retrabalho e perda de qualidade pedagógica.

É nesse contexto que se encaixa a proposta deste trabalho. As ferramentas de produtividade que existem hoje, como Trello, Notion e Asana, foram feitas para o trabalho de empresas em geral. Elas não diferenciam uma “aula” de uma “tarefa administrativa”, nem um “roteiro” de uma “nota solta”. Por não terem uma estrutura própria para a educação, o criador precisa adaptar essas ferramentas o tempo todo, gastando tempo e energia que deveriam ir para a qualidade do ensino. O Creator Workbench surge como resposta a esse problema: uma plataforma feita para a rotina de quem cria conteúdo educacional digital.

1.1 OBJETIVOS

Esta seção apresenta o objetivo geral e os objetivos específicos do desenvolvimento do Creator Workbench, ou seja, o que se pretende alcançar com a plataforma e

quais funcionalidades são necessárias para isso.

1.1.1 Objetivo Geral

Desenvolver uma plataforma web para reunir e melhorar o *pipeline* de produção de conteúdo educacional, permitindo passar de ideias soltas para cursos organizados em módulos e aulas.

1.1.2 Objetivos Específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- Implementar um módulo de gestão de ideias, com votação da equipe para escolher o que produzir primeiro;
- Desenvolver um fluxo de trabalho (*workflow*) em um quadro *Kanban*, com etapas (Pesquisa, Roteiro, Gravação e Edição), tarefas e revisão dos projetos;
- Organizar os materiais de apoio, ligando cada arquivo ao projeto a que ele pertence;
- Criar uma funcionalidade para organizar o conteúdo em cursos, módulos e aulas, com publicação e exportação;
- Oferecer um painel de alertas de prazo e o acompanhamento do progresso das tarefas.

1.2 PROBLEMA E SOLUÇÃO PROPOSTA

O problema central é a desorganização do trabalho, que fica espalhado em várias ferramentas. Em 1969, Peter Drucker já dizia que tornar o trabalho do conhecimento produtivo seria o grande desafio da gestão no século XX, assim como tornar o trabalho manual produtivo foi o desafio do século anterior (DRUCKER, 1999). Para o criador de conteúdo educacional, essa falta de produtividade aparece no uso de ferramentas que não se conversam: aplicativos de notas, planilhas e pastas de arquivos sem padrão, que tomam tempo e atenção que deveriam ir para a qualidade pedagógica.

A solução proposta, o Creator Workbench, é um ambiente único que trata o conteúdo educacional como dados organizados. Nele, o roteiro, os materiais de apoio e os prazos ficam no mesmo lugar, o que reduz a confusão causada por tantas ferramentas diferentes.

1.3 JUSTIFICATIVA

A importância do projeto vem do crescimento rápido do setor. Segundo a Goldman Sachs, o mercado da *creator economy* pode dobrar de tamanho em cinco anos, passando de US$ 250 bilhões para US$ 480 bilhões até 2027 (Goldman Sachs, 2023). Com isso, cresce o número de criadores que precisam organizar a própria produção, inclusive na área da educação.

A importância desta solução se apoia em dois pontos. O primeiro é o alcance como projeto de extensão, que atende desde o professor autônomo até agências de lançamento que cuidam do trabalho de vários especialistas. O segundo é o efeito direto na qualidade do trabalho criativo: pesquisas mostram que o excesso de tecnologia causado pelo uso de muitas ferramentas digitais leva ao *burnout* e reduz a criatividade de trabalhadores de plataformas (BUNJAK; ČERNE; POPOVIč, 2021). Isso pesa ainda mais para quem cria conteúdo educacional, porque o valor entregue depende da criatividade e da clareza pedagógica. O Creator Workbench ajuda a profissionalizar o setor, permitindo que o criador use sua energia para ensinar, e não para administrar arquivos.

1.4 ANÁLISE DA CONCORRÊNCIA

Esta seção apresenta as ferramentas de produtividade que o criador de conteúdo educacional usa hoje para organizar sua produção e os critérios usados para escolhê-las.

1.4.1 Critérios de seleção

Para evitar escolhas por opinião, a seleção dos concorrentes seguiu critérios que podem ser conferidos em fontes oficiais. Foram incluídas as ferramentas que atendem às quatro condições abaixo ao mesmo tempo: (i) são de uso geral, ou seja, não têm estrutura própria para a área educacional; (ii) têm plano gratuito ou de entrada com preço público; (iii) têm interface em português do Brasil; e (iv) não exigem um número mínimo de usuários que impeça o uso individual. Também se buscou representar três jeitos diferentes de organizar o trabalho: quadros *Kanban* (Trello), documentos e bancos de dados flexíveis (Notion) e gestão de projetos por tarefas (Asana).

As outras ferramentas citadas no setor ficaram de fora pelos seguintes motivos. O monday.com exige no mínimo três usuários nos planos pagos, o que não serve para o criador que trabalha sozinho. O Craft não é uma ferramenta de gestão de projetos, tem poucas integrações e não tem versão para *Android*. No Coda, não foi possível confirmar em fonte oficial que a interface é em português. O ClickUp atende

aos critérios, mas funciona de forma parecida com o Asana, por isso foi tratado como equivalente a ele. Essa escolha é uma limitação deste estudo.

1.4.2 Trello

O Trello é uma ferramenta visual baseada em *Kanban*. O plano gratuito permite até 10 colaboradores e 10 quadros por espaço de trabalho, e o plano *Standard* custa US$ 5,00 por usuário ao mês na cobrança anual (Atlassian, 2026). Ele funciona bem para fluxos simples, mas não tem campos próprios para roteiro nem permite organizar hierarquias mais complexas, como os módulos de um curso.

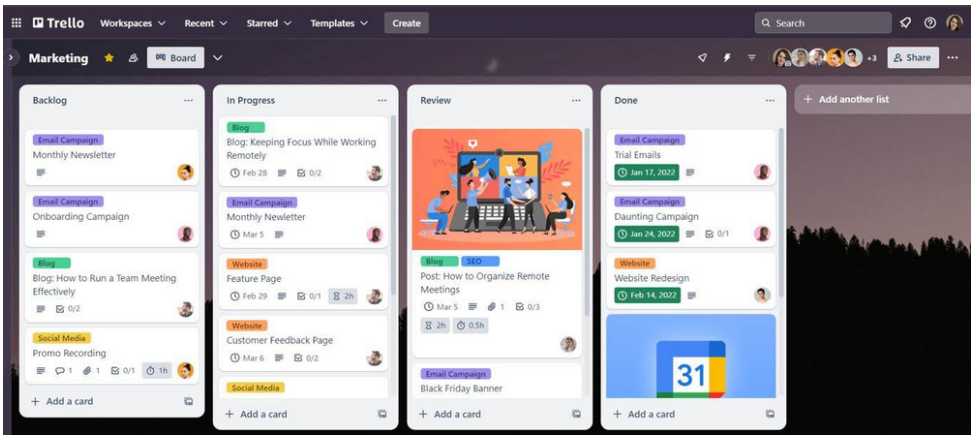


Figura 1 – Interface da plataforma Trello

Fonte: Elaborado pelos autores (2026).

1.4.3 Notion

O Notion é uma plataforma de documentos e bancos de dados muito flexível. Ele tem plano gratuito, e o plano *Plus* custa US$ 10,00 por membro ao mês na cobrança anual (Notion Labs, 2026). Como não tem estrutura própria para cursos, o usuário precisa criar e manter os seus próprios bancos de dados, o que toma tempo de configuração.

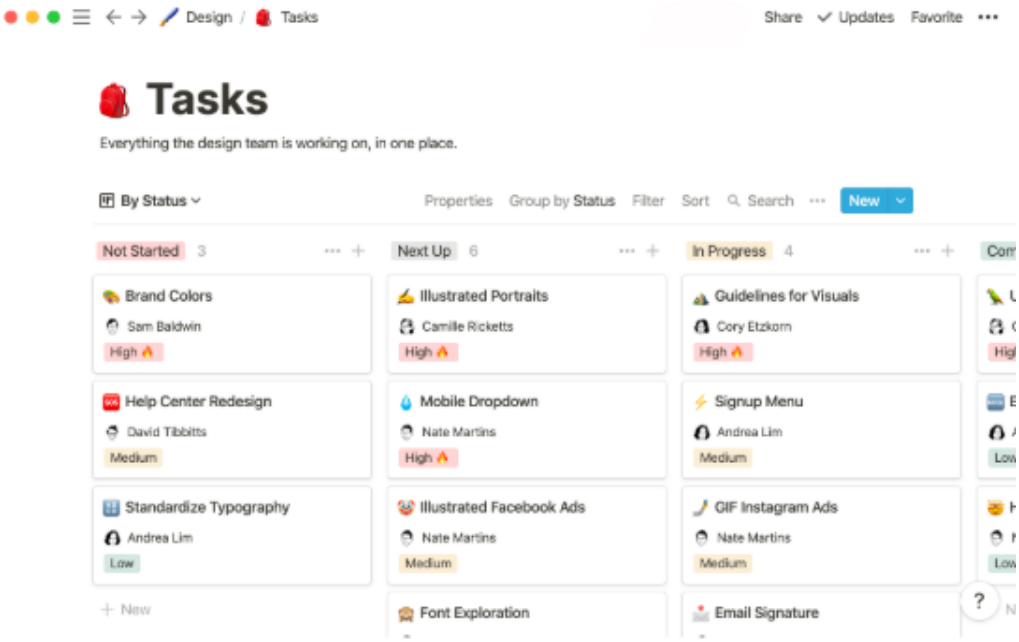


Figura 2 – Interface da plataforma Notion

Fonte: Elaborado pelos autores (2026).

1.4.4 Asana

O Asana é voltado à gestão de projetos por tarefas. O plano *Starter* custa US$ 10,99 por usuário ao mês na cobrança anual (Asana, Inc., 2026). Ele traz recursos pensados para equipes, como o controle da carga de trabalho de cada pessoa, que servem pouco ao criador individual, e não foi feito para o fluxo de produção de cursos.

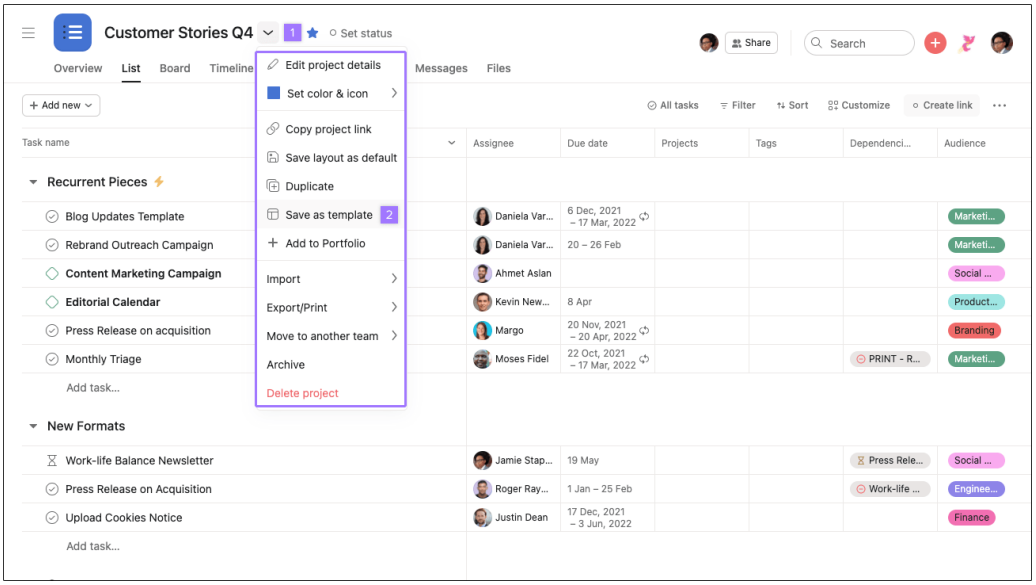


Figura 3 – Interface da plataforma Asana

Fonte: Elaborado pelos autores (2026).

1.4.5 Comparativo

A Tabela 1 compara as plataformas. Os preços em reais foram convertidos pela cotação PTAX de venda de 28/10/2026 (R$ 5,3420) (Banco Central do Brasil, 2026). O preço do Creator Workbench é o do plano pago mais barato (Solo), com cobrança mensal. Os preços dos concorrentes são os da cobrança anual.

Tabela 1 – Comparativo entre plataformas (consulta em 03/11/2026)

Recurso	**Trello**	**Notion**	**Asana**	**Creator Workbench**
Foco Educacional	Baixo	Médio	Baixo	Alto
Estrutura de Cursos	Não	Manual	Não	Nativo
Gestão de Materiais	Link externo	Interno	Link externo	Integrado
Custo (plano pago básico, por usuário/mês)	US$ 5,00 (R$ 26,71)	US$ 10,00 (R$ 53,42)	US$ 10,99 (R$ 58,71)	R$ 49,00 (plano Solo)
Plano gratuito	Sim	Sim	Sim	Sim (limitado)
Interface em português (Brasil)	Sim	Sim	Sim	Sim
Campos próprios para roteiro	Não	Manual	Não	Nativos

Fonte: Elaborado pelos autores com base em (Atlassian, 2026; Notion Labs, 2026; Asana, Inc., 2026; Banco Central do Brasil, 2026).

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo explica as ideias que servem de base para o projeto: como a gestão de conteúdo digital surgiu e mudou ao longo do tempo, como ela funciona hoje e quais teorias ajudam a entender o problema.

2.1 HISTÓRICO DO ASSUNTO

A gestão de conteúdo digital vem dos arquivos de papel e mudou bastante com o surgimento dos *Content Management Systems* (CMS), na década de 1990. Segundo Boiko (BOIKO, 2004), um CMS é uma aplicação *web* que permite que pessoas sem formação técnica criem, atualizem e publiquem conteúdo digital. Pela primeira vez, escrever o conteúdo ficou separado de programar como ele aparece na tela. Essa separação facilitou a criação de conteúdo na *web* e deu origem aos sistemas de gestão que existem hoje.

Essa mudança cresceu com a popularidade das plataformas de vídeo e dos ambientes de educação *online*, como Coursera e Udemy. Para Rosenfeld, Morville e Arango (ROSENFELD; MORVILLE; ARANGO, 2015), a Arquitetura da Informação serve para que os ambientes digitais sejam fáceis de encontrar, de navegar e de entender. Isso fica mais importante quando o conteúdo cresce e fica mais complexo. Por isso, ferramentas de organização feitas para a área da educação deixam de ser um diferencial e passam a ser uma necessidade.

Essa complexidade afeta principalmente quem cria conteúdo sozinho. Diferente de uma empresa, onde cada pessoa tem uma função, o criador de cursos digitais faz o papel de roteirista, produtor, editor e gestor do projeto ao mesmo tempo. Como diz Drucker (DRUCKER, 1999), o trabalhador do conhecimento decide sozinho qual é a sua tarefa. Por isso, ele precisa de liberdade, mas também de organização, para não perder o foco. Sem um sistema que siga a estrutura natural de um curso (em que aulas dependem de roteiros, roteiros dependem de pesquisas e grupos de aulas formam módulos), o trabalho se espalha entre aplicativos de notas, planilhas e pastas de arquivos sem padrão.

2.2 ATUALIDADE DO ASSUNTO

Hoje, muitas empresas usam o que se chama de *Content Operations* (ContentOps). Segundo o Content Marketing Institute (MCKNIGHT, 2023), o termo se refere a tudo o que envolve conteúdo dentro de uma organização: o planejamento, a criação, o controle de qualidade, a medição de resultados e a organização do material. A mesma

fonte explica que a ideia é que todos produzam conteúdo seguindo um mesmo jeito de trabalhar, em vez de cada pessoa ter o seu próprio processo.

Na parte técnica, esse movimento usa sistemas *headless*, que separam a criação do conteúdo da forma como ele aparece na tela. Assim, o conteúdo é feito uma única vez e pode ser publicado em vários canais, como site, aplicativo e redes sociais. Esse modelo é chamado de *omnichannel*. O Gartner previu que, até 2026, pelo menos 70% das organizações precisariam adotar plataformas de experiência digital montadas em partes (DEMENY, 2025).

Para quem cria conteúdo sozinho, porém, essas práticas ainda são pouco comuns. As soluções existentes foram feitas para empresas grandes, e faltam ferramentas pensadas para quem trabalha sozinho ou em equipes pequenas. É essa lacuna que o Creator Workbench quer preencher.

2.3 OUTROS CONTEXTOS DO ASSUNTO

Dois campos de estudo ajudam a entender o problema deste trabalho: a Andragogia e a Arquitetura da Informação.

2.3.1 Andragogia

A Andragogia é definida por Malcolm Knowles, seu principal autor, como a arte e a ciência de ensinar adultos. Na obra *The Modern Practice of Adult Education: Pedagogy vs. Andragogy* (KNOWLES, 1970), Knowles defende que a educação de adultos precisa ser diferente da educação de crianças. Entre os princípios que ele aponta estão a necessidade de saber por que está aprendendo, a autonomia de quem aprende, o uso da experiência anterior e o foco em resolver problemas.

Nos cursos digitais, essa teoria ajuda a pensar no conteúdo. Um curso organizado de forma clara, em ordem e com ligações visíveis entre suas partes ajuda o aluno a aprender com mais autonomia e a guardar melhor o que aprendeu. Ao organizar módulos e aulas de forma nativa, o Creator Workbench dá ao criador as condições técnicas para aplicar esses princípios no seu curso.

Em *The Adult Learner* (KNOWLES; HOLTON; SWANSON, 2015), publicada no Brasil como *Aprendizagem de Resultados*, Knowles apresenta os seis princípios da aprendizagem de adultos. A obra é um clássico da educação em empresas e traz pontos que devem ser considerados em qualquer ação educacional.

Entre esses princípios, dois combinam bem com este projeto: aprender para resolver problemas e ver utilidade imediata no conteúdo. Um criador que não consegue organizar o próprio trabalho dificilmente vai montar um curso que respeite esses princípios para os seus alunos.

2.3.2 Arquitetura da Informação

A Arquitetura da Informação ajuda a pensar como organizar os sistemas de criação. Em *Information Architecture: For the Web and Beyond* (ROSENFELD; MORVILLE; ARANGO, 2015), Rosenfeld, Morville e Arango explicam que a área busca tornar os ambientes digitais fáceis de encontrar, de navegar e de entender. Seus conceitos principais são organização, rotulação, navegação e busca.

No trabalho com conteúdo, essa ideia mostra por que ferramentas genéricas, como Trello e Notion, muitas vezes não funcionam bem para cursos: elas não têm uma estrutura própria para a educação. Um *board* de *Kanban* não diferencia uma "aula" de uma "tarefa administrativa". Um banco de dados do Notion, por padrão, não tem a ideia de "dependência entre roteiros". O Creator Workbench coloca essa estrutura dentro do próprio sistema e aplica as ideias de Rosenfeld et al. à criação de cursos digitais. Assim, o sistema fica organizado e também fácil de entender para quem o usa.

3 GESTÃO DO PROJETO

Este capítulo mostra como a equipe se organiza, qual metodologia usa para trabalhar e onde o código do projeto fica guardado.

3.1 ORGANIZAÇÃO DA EQUIPE

Como o projeto é acadêmico, a equipe trabalha em um modelo misto: cada integrante tem um papel principal, mas todos ajudam nas outras áreas quando necessário. Os papéis e as responsabilidades de cada pessoa estão descritos a seguir.

3.1.1 Responsabilidades, Papéis e Atividades

- **Líder Técnico e Engenheiro de *Backend* (*Tech Lead*): Luciano Ricardo Paim de Araujo.** É a pessoa que decide sobre a arquitetura do sistema. Cuida da API REST (Java/Spring Boot), da segurança (JWT), da integração entre os serviços e da arquitetura na AWS. Também define os padrões de código e revisa os *Pull Requests* mais importantes.
- ***Product Owner* (PO) e Analista de Documentação: Vitoria de Lima Santos.** Define o *Backlog*, escolhe a ordem das funcionalidades e cuida da escrita acadêmica (ABNT). Faz o papel de "cliente" do projeto, conferindo se o que foi entregue atende aos requisitos pedagógicos e de negócio.
- **Engenheiro de *Front-end*: Henrique Orsoni.** Desenvolve a *Single Page Application* (Next.js e React). Garante que a tela fique igual ao projeto visual (UI/UX), cuida do estado global da aplicação e do consumo correto dos *endpoints* da API.
- **DBA / Arquiteto de Dados: Renato Vanil de Oliveira.** Faz a modelagem do banco de dados relacional (MySQL), cria os índices, aplica a normalização (3FN), garante a integridade das ligações entre as tabelas e cuida das migrações do banco.
- **Engenheiro de Automação e *DevOps*: Augusto Henrique Braga Silva.** Cuida do provisionamento da infraestrutura na AWS (Amazon EKS, ECR, RDS, S3 e CloudWatch), do empacotamento em contêineres Docker, do serviço de automação em Python e da integração contínua com GitHub Actions.

3.2 METODOLOGIAS DE GESTÃO E DESENVOLVIMENTO

Para organizar o trabalho, a equipe usa o *Scrum*, que é explicado a seguir.

3.2.1 Scrum

O *Scrum* foi adotado como metodologia de desenvolvimento porque o projeto cria um produto digital novo, e nesse tipo de trabalho muita coisa ainda não é conhecida no começo. Segundo Schwaber e Sutherland (SCHWABER; SUTHERLAND, 2020), criadores do *framework*, o *Scrum* trabalha em ciclos curtos e em partes, para prever melhor os resultados e controlar os riscos. Ele se apoia em três pilares: transparência, inspeção e adaptação.

Essa forma de trabalhar ajuda no desenvolvimento do Creator Workbench, porque as funcionalidades são entregues em ciclos curtos, chamados *Sprints*. Assim, é possível testar com usuários reais e ajustar os requisitos conforme o produto evolui. Os autores também explicam que o *Scrum* não é um processo fechado, e sim um *framework* dentro do qual cada equipe pode usar as técnicas de que precisa. Por isso ele combina bem com um projeto que começa por um MVP (Produto Mínimo Viável) e precisa mudar de acordo com o retorno do mercado.

3.2.1.1 *Sprints*

O projeto foi organizado em *Sprints* quinzenais, com o total de 8 *Sprints* no cronograma. Em cada *Sprint*, a equipe entrega uma parte funcional do sistema, o que permite testar a usabilidade o tempo todo. O andamento das tarefas de cada ciclo é acompanhado no quadro *Kanban* do GitHub Projects, e o histórico de *commits* da *branch* principal registra o que foi entregue.

A Tabela 2 resume as entregas de cada *Sprint* e a situação atual.

Tabela 2 – Entregas por *Sprint* e situação atual

Sprint	**Entregas**	**Situação**
1	Infraestrutura na AWS e base do *front-end* (Next.js 16, TypeScript, Tailwind CSS 4 e shadcn/ui)	Concluída
2	Autenticação com JWT e cadastro em duas etapas com código enviado por e-mail	Concluída
3	Quadro *Kanban* com quatro colunas e conversão de ideia em projeto	Concluída
4	Estrutura curricular com módulos e aulas e controle de publicação	Concluída
5	Editor de texto rico e versionamento de roteiros	Concluída
6	Perfil do usuário, temas de cores e modo escuro	Concluída
7	Integração do *front-end* com a API, proteção das rotas e documentação técnica	Concluída
8	Equipes e papéis, etiquetas e votação, quadro configurável com tarefas e prazos, cursos com módulos e aulas, recuperação de senha, materiais, revisão, resumo automático, alertas de prazo, publicação e exportação de cursos, convites, planos e implantação em contêineres no Amazon EKS	Concluída

Fonte: Elaborado pelos autores (2026), com base no GitHub Projects do repositório.

3.3 REPOSITÓRIO DA APLICAÇÃO

O código-fonte e os arquivos do projeto ficam guardados em repositórios online, descritos a seguir.

3.3.1 Definição do Repositório

O projeto usa o **GitHub** para guardar o código. Dele aproveitamos o *GitHub Actions*, que executa os testes automaticamente, e o *Project Boards*, que controla as tarefas.

3.3.2 Link do Repositório

O código está dividido em dois repositórios, um para o *front-end* e outro para o *back-end*.

- ***Front-end*:** <https://github.com/HenriqueOrsoni/Creator-Workbench>
- ***Back-end*:** <https://github.com/luciano-araujo/ifsp-projeto-extensao>
- **Acesso:** o repositório é público para leitura, e as *branches* de produção são protegidas.

4 DESENVOLVIMENTO DO PROJETO

Este capítulo descreve o que o sistema faz, como ele foi organizado e quais tecnologias, testes, regras de segurança, banco de dados e prazos fazem parte do projeto.

4.1 ESCOPO DO PROJETO

O escopo compreende o desenvolvimento de uma SPA (*Single Page Application*) integrada a uma API RESTful. Nesta seção estão as regras de negócio, os requisitos funcionais e os requisitos não funcionais.

4.1.1 Regras de Negócio (RN)

As regras de negócio dizem como o sistema deve se comportar em cada situação do dia a dia do criador de conteúdo. As seis primeiras vêm do levantamento inicial do projeto.

- **RN01 – Fluxo de Aprovação:** uma “Ideia” só pode ser convertida em “Projeto de Conteúdo” depois que o público-alvo e o objetivo pedagógico forem preenchidos. O sistema recusa a conversão com campos vazios ou curtos demais: o título e o público-alvo precisam ter pelo menos 3 caracteres, e o objetivo, pelo menos 10.
- **RN02 – Versionamento de Roteiro:** o roteiro mantém um histórico de versões. Cada versão salva é uma cópia completa do texto e pode ser restaurada.
- **RN03 – Consistência de Grade:** um curso não pode ser publicado se não tiver módulos, se tiver módulos sem aulas, se tiver aulas sem material ou se tiver aulas com link inválido. Uma aula está sem material quando não tem nenhum material ligado a ela e nenhum link de conteúdo. Enquanto houver problemas, a publicação fica bloqueada.
- **RN04 – Gestão de Armazenamento:** uma rotina diária move para o Amazon S3 Glacier (classe *Instant Retrieval*) os materiais de cursos publicados há mais de 30 dias. Isso reduz o custo de armazenamento sem impedir o *download*.
- **RN05 – Unicidade de Identificação:** cada conteúdo (ideia ou projeto, roteiro, material, curso, módulo e aula) recebe um UUID (*Universally Unique Identifier*) na criação. A API devolve esse identificador, que permite a *scripts* e ferramentas externas acompanhar o mesmo conteúdo da ideia até a publicação.

- **RN06 – Regra de Status:** um curso só é publicado se os links das aulas forem válidos. A verificação de publicação valida cada link por expressão regular (*Regex*). São aceitos links do YouTube, do Vimeo e links HTTPS de plataformas de ensino (LMS) cadastradas na configuração do sistema, como Moodle e Google Classroom.

Detalhamento da RN06. As expressões regulares usadas na validação são as seguintes:

```
[PREENCHER: expressão regular do YouTube, copiada do código]
[PREENCHER: expressão regular do Vimeo, copiada do código]
[PREENCHER: expressão regular dos links de LMS cadastrados]
```

Exemplos de links **válidos**: [PREENCHER: um link do YouTube, um do Vimeo e um de LMS cadastrado]. Exemplos de links **inválidos**: [PREENCHER: um link de plataforma não cadastrada e um link sem *https*].

Durante o desenvolvimento, a equipe definiu outras regras que o sistema também aplica:

- **Equipe:** toda equipe tem pelo menos um Administrador. O último Administrador não pode ser removido, rebaixado nem sair sem passar a administração para outra pessoa.
- **Votos:** cada membro vota uma única vez em cada ideia. Um segundo clique retira o voto.
- **Revisão:** pedir ajustes exige um comentário. Ideias que ainda não viraram projeto não recebem revisão.
- **Convites:** o convite vale 7 dias e é pessoal. Só a conta com o mesmo e-mail pode aceitá-lo.
- **Códigos por e-mail:** os códigos valem 10 minutos e aceitam até 5 tentativas.
- **Resumo automático:** o resumo exige um roteiro com pelo menos 100 palavras.
- **Materiais:** os arquivos podem ter até 20 MB e precisam ser de um dos tipos permitidos.

4.1.2 Requisitos Funcionais (RF)

Os requisitos funcionais dizem o que o sistema faz. Eles estão agrupados por módulo, para facilitar a leitura.

Módulo de Autenticação e Perfil

- **RF01:** cadastrar usuários em duas etapas. O usuário informa nome e e-mail, recebe um código de 6 dígitos por e-mail e, com ele, define a senha. O sistema também permite recuperar a senha por código, trocar o e-mail e excluir a conta.
- **RF02:** controlar o acesso por funções (RBAC) em cada equipe: Administrador, Criador e Revisor. A mesma pessoa pode ter papéis diferentes em equipes diferentes. A Tabela 3 mostra o que cada papel pode fazer.

Tabela 3 – Permissões por papel na equipe

Ação	**Administrador**	**Criador**	**Revisor**
Ver quadro, ideias, roteiros, materiais e cursos	Sim	Sim	Sim
Votar em ideias e marcar tarefas como feitas	Sim	Sim	Sim
Criar ideias, converter em projeto, mover e excluir *cards*	Sim	Sim	Não
Criar, editar e atribuir tarefas	Sim	Sim	Não
Escrever e versionar roteiros, gerar resumo, enviar materiais	Sim	Sim	Não
Criar, editar, publicar e exportar cursos	Sim	Sim	Não
Registrar revisão (aprovar ou pedir ajustes)	Sim	Não	Sim
Gerenciar colunas e etiquetas	Sim	Não	Não
Convidar, remover membros e mudar papéis	Sim	Não	Não
Renomear e excluir a equipe	Sim	Não	Não
Sair da equipe	Sim	Sim	Sim

Módulo de Gestão de Ideias (Inbox)

- **RF03:** permitir a criação de *cards* de ideias com *tags* coloridas. Cada equipe tem a sua lista de etiquetas, que começa com Tutorial, Podcast, Curso, Roteiro e Artigo.
- **RF04:** ter um sistema de votação para priorizar as ideias em equipe. O *card* mostra o total de votos, e a Caixa de Entrada pode ser ordenada por votos.

Módulo de Produção (*Workflow*)

- **RF05:** oferecer um quadro *Kanban* com colunas configuráveis por equipe. As colunas padrão são Pesquisa, Roteiro, Gravação e Edição. Cada projeto tem tarefas com responsável e prazo, e o progresso é calculado automaticamente pelas tarefas concluídas.
- **RF06:** permitir o *upload* de materiais de apoio (PDF, Word, PowerPoint, Excel, texto e imagens PNG, JPG, WEBP e GIF) de até 20 MB, ligados ao projeto e guardados no Amazon S3.

- **RF07:** oferecer um editor de texto rico (*Rich Text*), chamado Estúdio de Escrita, para escrever roteiros dentro da plataforma, com histórico de versões.

Módulo de Automação

- **RF08:** gerar um resumo do conteúdo do projeto a partir do roteiro. O *back-end* envia o texto do roteiro a um serviço de automação escrito em Python, que devolve o resumo.
- **RF09:** mostrar um painel de alertas com os projetos e as tarefas atrasados ou que vencem nos próximos 3 dias. Todos os dias, às 8h (horário de Brasília), cada pessoa recebe um único e-mail com os prazos vencidos e próximos de todas as suas equipes.

Módulo de Estruturação Curricular (LMS *Ready*)

- **RF10:** permitir a criação de cursos e a organização dos conteúdos em módulos e aulas (vídeo, texto ou questionário).
- **RF11:** exportar a estrutura do curso em JSON ou CSV, para usar em plataformas externas.

Além dos requisitos acima, o sistema oferece a revisão de projetos (aprovar ou pedir ajustes), os convites por e-mail para novos membros, a gestão da equipe, a gestão da conta do usuário e o controle dos planos de assinatura, com um plano Gratuito e um teste grátis de 14 dias dos planos pagos.

4.1.3 Requisitos Não Funcionais (RNF)

Os requisitos não funcionais dizem como o sistema funciona: com que segurança, com que velocidade e com que facilidade de manutenção.

Segurança e Confiabilidade

- **RNF01:** as senhas são criptografadas com o algoritmo BCrypt e nunca aparecem nas respostas da API.
- **RNF03:** a autenticação usa *tokens* JWT assinados, válidos por 24 horas e enviados no cabeçalho *Authorization*.

Performance e Escalabilidade

- **RNF05:** os e-mails (códigos, convites e alertas) são enviados de forma assíncrona, sem atrasar a resposta da API.

- **RNF06:** o banco de dados está normalizado até a 3ª Forma Normal (3FN), com chaves estrangeiras e regras de exclusão em cascata.

Infraestrutura e Manutenibilidade

- **RNF07:** o *back-end* tem testes unitários com JUnit 5 e Mockito, e a cobertura é medida pelo JaCoCo. A meta é acima de 70%.
- **RNF08:** a documentação da API é gerada automaticamente pelo Swagger (OpenAPI), com o springdoc.
- **RNF09:** o *front-end*, o *back-end* e o serviço de automação em Python rodam em contêineres Docker, com imagens no Amazon ECR, orquestrados pelo Amazon EKS.
- **RNF10:** os *logs* dos contêineres são enviados ao Amazon CloudWatch Logs.

4.2 HISTÓRIAS DE USUÁRIO

As histórias de usuário descrevem o que cada tipo de pessoa quer fazer no sistema. Aqui elas são apresentadas, detalhadas em critérios de aceitação que podem ser testados e ligadas aos cenários de teste.

4.2.1 Descrição das Histórias de Usuário

As histórias seguem o formato padrão *“Como [perfil], quero [ação], para [valor]”* e vêm dos requisitos funcionais.

Tabela 4 – Histórias de Usuário

ID	Perfil	História	Cenários de aceitação
US01	Criador	Como criador, quero capturar uma ideia rapidamente com título e descrição, para não perder *insights* durante o processo criativo.	CT01, CT02
US02	Criador	Como criador, quero mover um conteúdo entre as etapas do *Kanban*, para acompanhar o progresso da produção.	CT03
US03	Criador	Como criador, quero ativar um projeto a partir de uma ideia preenchendo o Público-Alvo e o Objetivo Pedagógico, para estruturar formalmente a produção.	CT04, CT05
US04	Criador	Como criador, quero fazer *upload* de arquivos PDF e imagens ligados a um projeto, para centralizar os materiais de apoio.	CT06, CT07
US05	Criador	Como criador, quero escrever o roteiro direto na plataforma com um editor de texto rico, para evitar o uso de ferramentas externas.	CT08, CT09
US06	Administrador	Como administrador, quero convidar colaboradores com perfis diferentes (Criador/Revisor), para controlar o acesso às funcionalidades.	CT10, CT11, CT12
US07	Criador	Como criador, quero organizar conteúdos em Módulos e Aulas dentro de um Curso, para estruturar o currículo pedagógico.	CT13, CT14, CT15
US08	Criador	Como criador, quero exportar a estrutura do curso em JSON ou CSV, para integrá-la a plataformas externas de LMS.	CT16, CT17

4.2.2 Critérios de Aceitação em Gherkin

Para que cada critério possa ser testado de forma objetiva, eles foram escritos no formato *Gherkin* (Dado/Quando/Então). Cada cenário recebe um código (CT), usado na tabela de rastreabilidade.

US01: Capturar ideia

CT01: Criar ideia com título. Dado que sou um criador autenticado e estou na Caixa de Entrada, **quando** preencho o título e salvo, **então** um *card* com esse título aparece na Caixa de Entrada.

CT02: Título obrigatório. Dado que estou criando uma ideia, **quando** tento salvar sem título, **então** o *card* não é criado e o sistema mostra uma mensagem de erro.

US02: Mover conteúdo no Kanban

CT03: Mudar de coluna. Dado que um projeto está na coluna Pesquisa, **quando** o movo para a coluna Roteiro, **então** o projeto aparece na nova coluna e o histórico do *card* registra a coluna, a data e quem fez a mudança.

US03: Ativar projeto

CT04: Campos vazios ou curtos (RN01). Dado que tenho uma ideia, **quando** tento convertê-la com o público-alvo vazio ou com o objetivo pedagógico de menos de 10 caracteres, **então** o sistema recusa a conversão e informa o campo com problema.

CT05: Conversão válida. Dado que informei título, público-alvo e objetivo pedagógico válidos e não escolhi coluna, **quando** confirmo a conversão, **então** o projeto é criado na primeira coluna do quadro.

US04: Enviar materiais

CT06: Envio válido (RF06). Dado que estou em um projeto, **quando** envio um arquivo PDF de até 20 MB, **então** o arquivo aparece na lista de materiais com nome, tipo, tamanho, autor e data.

CT07: Arquivo recusado. Dado que estou em um projeto, **quando** envio um arquivo de tipo não permitido ou maior que 20 MB, **então** o envio é recusado e o sistema mostra uma mensagem de erro.

US05: Escrever roteiro

CT08: Editor de texto rico (RF07). Dado que abri o Estúdio de Escrita de um projeto, **quando** escrevo um roteiro com negrito e lista e salvo, **então** ao abrir de novo o roteiro mantém a formatação.

CT09: Restaurar versão (RN02). Dado que salvei duas versões do roteiro, **quando** restauro a primeira, **então** o texto do editor volta a ser o da primeira versão.

US06: Convidar colaboradores

CT10: Convite com papel (RF02). Dado que sou Administrador, **quando** convido um e-mail com o papel Revisor, **então** o convite fica pendente por 7 dias e, ao ser aceito pela conta com esse e-mail, a pessoa entra na equipe como Revisor.

CT11: Ação sem permissão. Dado que sou Revisor, **quando** tento mover um *card*, **então** a API responde com o código 403 e a mensagem “Seu papel na equipe não permite esta ação”.

CT12: Último administrador. Dado que sou o único Administrador da equipe, **quando** tento sair ou me rebaixar, **então** o sistema recusa e pede que eu promova outra pessoa antes.

US07: Organizar o curso

CT13: Hierarquia do curso (RF10). Dado que tenho um curso, **quando** crio um módulo e adiciono aulas a ele, **então** a hierarquia Curso $\rightarrow$ Módulo $\rightarrow$ Aula é salva e mostrada na ordem criada.

CT14: Publicação bloqueada (RN03). Dado que um curso tem um módulo sem aulas, **quando** tento publicá-lo, **então** o sistema lista o problema e a publicação é bloqueada.

CT15: Link inválido (RN06). Dado que um curso tem uma aula com um link que não é do YouTube, do Vimeo nem de um LMS cadastrado, **quando** tento publicá-lo, **então** o sistema lista a aula com link inválido e a publicação é bloqueada.

US08: Exportar o curso

CT16: Exportar em JSON (RF11). Dado que tenho um curso estruturado, **quando** exporto em JSON, **então** o arquivo traz o curso, os módulos, as aulas e os materiais.

CT17: Exportar em CSV (RF11). Dado que tenho um curso estruturado, **quando** exporto em CSV, **então** o arquivo, separado por ponto e vírgula e em UTF-8, traz uma linha

para cada aula, com os dados do módulo a que ela pertence.

4.2.3 Rastreabilidade entre Histórias de Usuário e Testes

A Tabela 5 mostra qual cenário de teste cobre cada história de usuário. O tipo de teste segue a pirâmide descrita na seção de testes.

Tabela 5 – Rastreabilidade entre Histórias de Usuário (US) e Cenários de Teste (CT)

US	**Requisito ou regra**	**Cenários**	**Tipo de teste**	**Situação**
US01	RF03	CT01, CT02	Unitário e integração	Aprovado
US02	RF05	CT03	Integração	Aprovado
US03	RN01	CT04, CT05	Unitário	Aprovado
US04	RF06	CT06, CT07	Integração	Aprovado
US05	RF07, RN02	CT08, CT09	Unitário e componente (Jest)	Aprovado
US06	RF02	CT10, CT11, CT12	Unitário e integração	Aprovado
US07	RF10, RN03, RN06	CT13, CT14, CT15	Unitário e integração	Aprovado
US08	RF11	CT16, CT17	Integração	Aprovado

4.3 ARQUITETURA

A arquitetura define como as partes do sistema se organizam e conversam entre si.

4.3.1 Definições da Arquitetura

O Creator Workbench adota o padrão *Client-Server* desacoplado (*Decoupled Architecture*). O *front-end* é uma aplicação de página única (SPA), feita em Next.js e publicada como arquivos estáticos. O *back-end* é uma API REST feita em Spring Boot. Os dois são independentes e conversam por HTTP, com dados em JSON, e cada requisição protegida leva o *token* JWT do usuário no cabeçalho *Authorization*.

Essa escolha traz três vantagens: (1) o *front-end* e o *back-end* podem ser publicados e versionados separadamente; (2) a API pode ser testada sozinha, com o Swagger gerado automaticamente; (3) novos clientes, como um aplicativo de celular, podem usar a mesma API sem mudar o servidor.

O *back-end* é dividido em camadas. O *controller* recebe a requisição HTTP. O caso de uso, uma classe para cada ação do sistema, guarda as regras de negócio. O serviço acessa os dados e monta as respostas, e o repositório faz as consultas ao banco. Essa divisão deixa cada regra em um único lugar e facilita os testes. Os dados

ficam em um banco MySQL, e a estrutura das tabelas é versionada por migrações Flyway.

Na infraestrutura, o sistema roda na AWS em contêineres Docker orquestrados pelo Amazon EKS (Kubernetes), dentro de uma VPC. São três contêineres: o *front-end* (Nginx servindo os arquivos estáticos do Next.js), o *back-end* (Spring Boot) e o serviço de automação em Python, que gera os resumos. As imagens ficam no Amazon ECR. Um Elastic Load Balancer recebe o acesso dos usuários e o entrega ao *front-end*, que encaminha as chamadas `/api` para o *back-end*. Os dados ficam no Amazon RDS (MySQL), acessível apenas de dentro da VPC. Os materiais enviados ficam no Amazon S3, e os de cursos publicados há mais de 30 dias passam para o S3 Glacier. Os *logs* dos contêineres vão para o Amazon CloudWatch, e os e-mails do sistema saem por um servidor SMTP.

4.3.2 Diagrama da Arquitetura

O diagrama abaixo mostra as partes do sistema e como elas se comunicam.

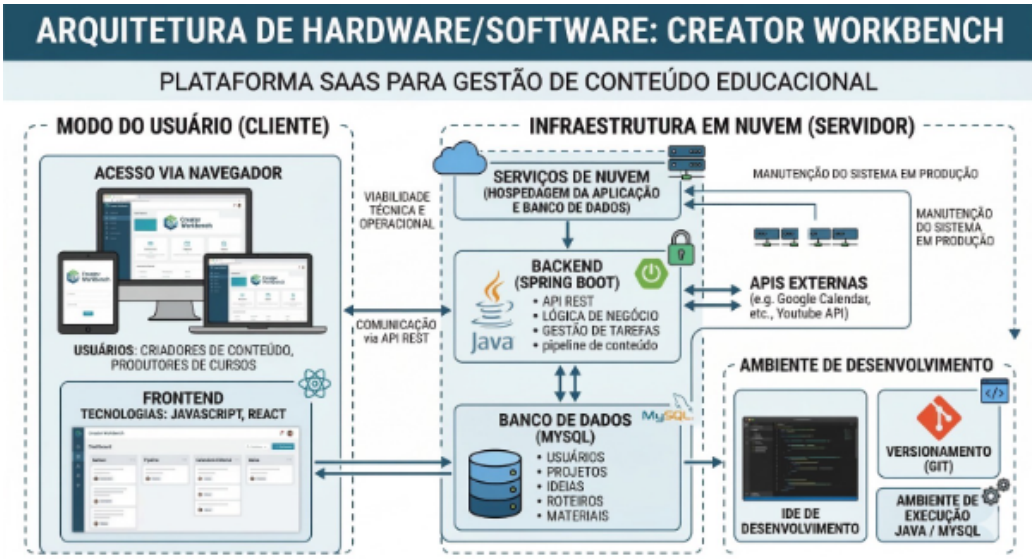


Figura 4 – Diagrama da Arquitetura Client-Server Desacoplado

Fonte: Produzido pelos autores.

4.3.3 Diagrama de Componentes

O diagrama de componentes mostra as partes internas de cada aplicação e como elas dependem umas das outras (visão *white-box*). A Figura 5 apresenta o *front-end*, o *back-end* e os serviços externos com que eles se comunicam.

No *front-end*, as telas e funcionalidades (autenticação, painel, ideias, quadro Kanban, detalhes do *card*, editor de roteiro, cursos, equipes e configurações do usuário) usam o módulo de estado (`useAppStore`) e o controle de acesso por papel

(`roles.ts`). O módulo de estado é quem chama a API, por meio do cliente HTTP (`api.ts`).

No *back-end*, toda requisição passa primeiro pelo módulo de segurança, que valida o *token* com o módulo JWT, e depois chega aos *controllers*. Os *controllers* chamam os casos de uso e serviços de cada módulo de domínio, e estes acessam o banco pela camada de repositório (Spring Data JPA/Hibernate), seguindo as camadas descritas na seção anterior. Alguns módulos falam com serviços de fora da aplicação: o Summary Module chama o serviço Python (`POST /summarize`), o Material Module guarda os arquivos no Amazon S3 e o Email Module envia os e-mails pelo servidor SMTP. As rotinas agendadas (*Scheduled Jobs*) rodam no próprio Spring Boot: apagam códigos de verificação vencidos, enviam o e-mail diário de prazos e arquivam os materiais antigos. Os três contêineres enviam seus *logs* ao Amazon CloudWatch.

4.3.4 Diagrama de Implantação

O diagrama de implantação mostra em que infraestrutura cada parte do sistema roda (Figura 6). O usuário acessa o sistema pelo navegador, no computador ou no celular, por HTTP. O acesso chega ao Elastic Load Balancing, dentro da VPC da AWS, que o entrega ao contêiner do *front-end*.

O *cluster* Amazon EKS tem três *namespaces*, um para cada contêiner: *front-end* (Nginx servindo o Next.js em exportação estática), *back-end* (Spring Boot) e automação (serviço em Python). O *front-end* encaminha as chamadas para o *back-end*, que por sua vez chama o serviço de automação. Fora do *cluster* ficam os serviços de apoio: o Amazon RDS (banco de dados MySQL), o Amazon S3 (arquivos e mídias), cujos materiais de cursos publicados há mais de 30 dias passam para a classe S3 Glacier, o Amazon ECR (imagens Docker), o Amazon CloudWatch (*logs*, métricas e alarmes) e o servidor SMTP, usado pelo *back-end* para os e-mails de cadastro, convites e alertas de prazo.

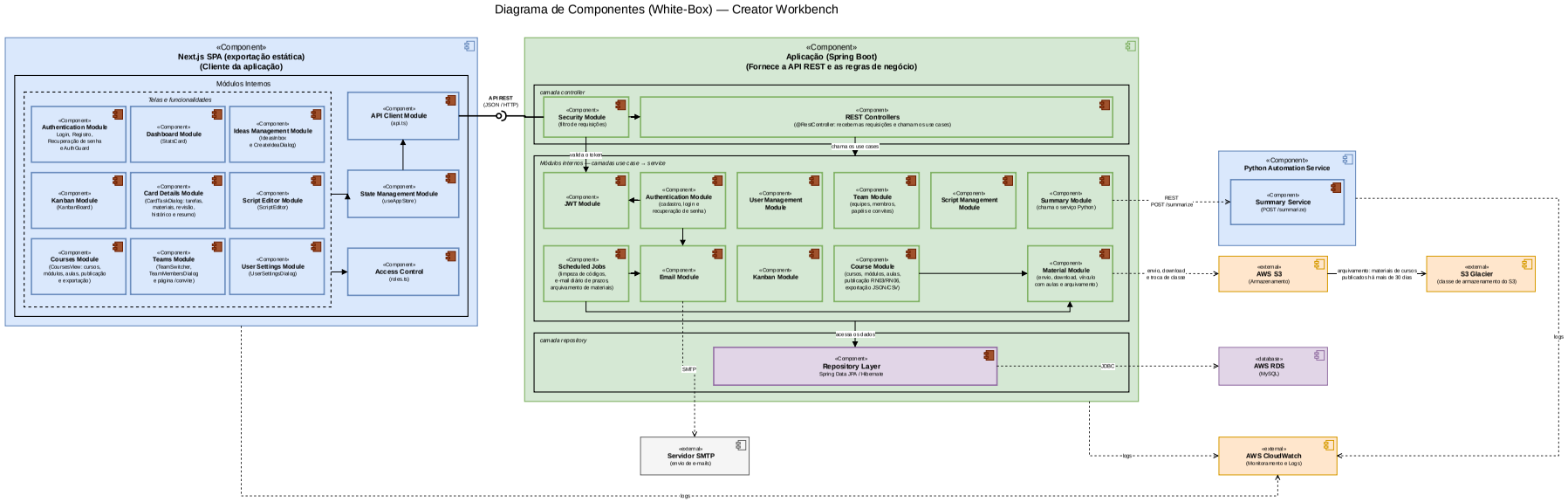
4.3.5 Integração com o Serviço de Automação

O requisito RF08 usa um serviço de automação escrito em Python, que roda em um contêiner próprio, para gerar o resumo do projeto. O resumo exige um roteiro com pelo menos 100 palavras. Com menos que isso, o *back-end* recusa o pedido antes de chamar o serviço e informa que não há dados suficientes. O último resumo gerado fica salvo no projeto, com a data de geração, e pode ser gerado de novo depois de mudanças no roteiro.

O contrato dessa comunicação, ou seja, o que cada lado envia e recebe, é o seguinte:

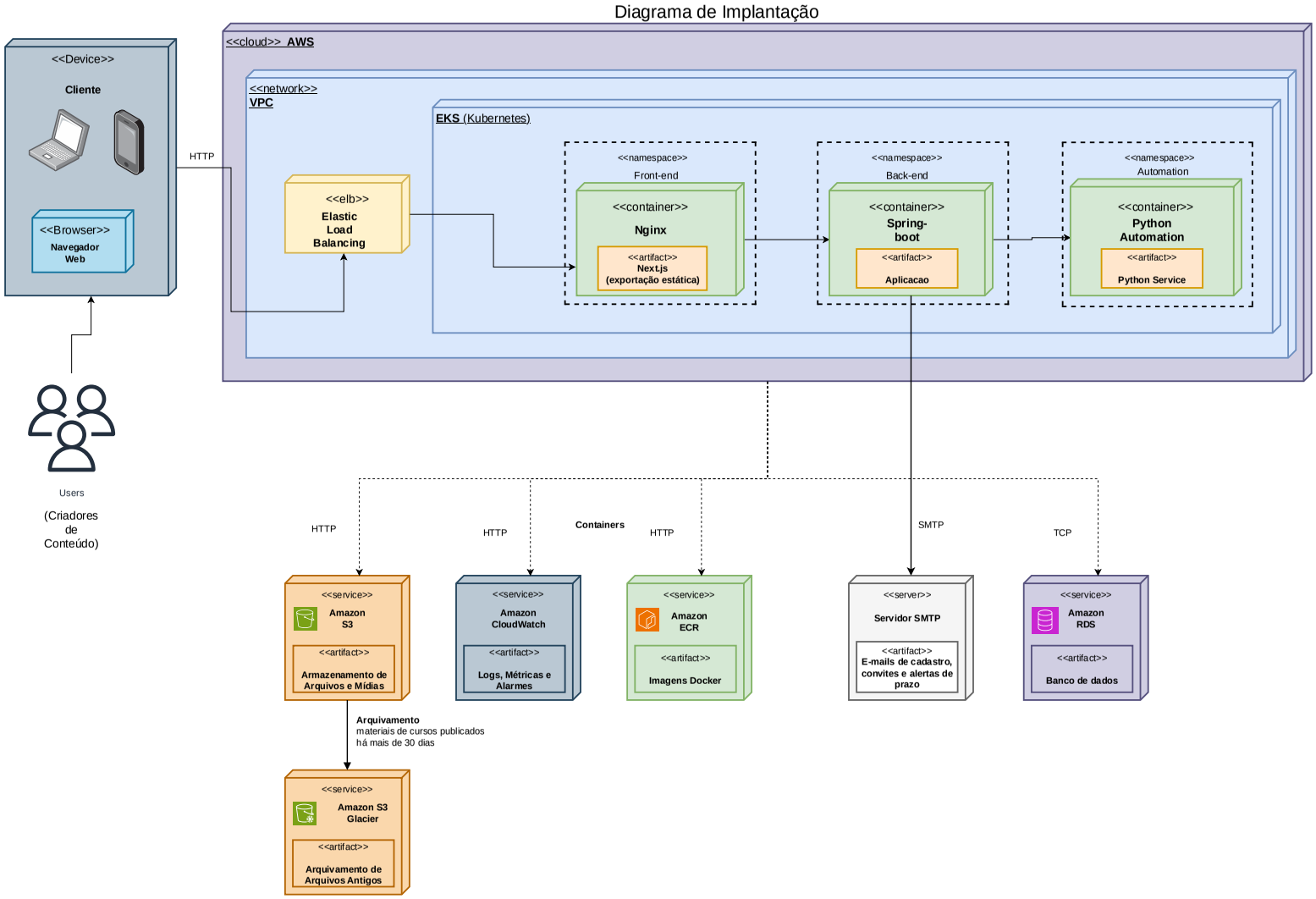
- **Mecanismo:** chamada REST direta, por HTTP, do *back-end* para o serviço

Figura 5 – Diagrama de Componentes (*White-Box*) - Creator Workbench



Fonte: Produzido pelos autores.

Figura 6 – Diagrama de Implantação - Creator Workbench



Fonte: Produzido pelos autores.

de automação em Python. Os dois rodam no mesmo *cluster*, e o serviço de automação não fica exposto à internet.

- **Tipo de chamada:** síncrona, com limite de 30 segundos.
- ***Endpoint*:** `POST /summarize` no serviço interno de automação.
- **Dados enviados:** um JSON com o texto do roteiro, sem formatação.
- **Dados recebidos:** um JSON com o texto do resumo.
- **Em caso de falha:** se o serviço não responder em 30 segundos ou devolver erro, a API responde com o código 503 e a mensagem “Não foi possível gerar o resumo agora. Tente novamente.”. O último resumo salvo continua no projeto, e o usuário vê a mensagem na tela.

Exemplo dos dados enviados e recebidos:

```
Pedido:   { "text": "<texto do roteiro, sem formatação>" }
Resposta: { "summary": "<texto do resumo>" }
```

4.4 TECNOLOGIAS

As tecnologias foram escolhidas para atender aos requisitos do projeto e à experiência da equipe.

4.4.1 *Front-end*

O *front-end* é uma *Single Page Application* (SPA) feita com Next.js e React, exportada como arquivos estáticos. Ele usa o *fetch* nativo do navegador para chamar os *endpoints* da API REST e o Zustand para guardar o estado global (equipe ativa, quadro, ideias e cursos).

Tabela 6 – Tecnologias de *Front-end*

Tecnologia	**Versão**	**Finalidade**
Next.js e React (TypeScript)	16 e 19	Aplicação de página única, exportada como arquivos estáticos
Tailwind CSS e shadcn/ui	4	Estilo e componentes da interface, com tema claro e escuro
Zustand	5	Estado global da aplicação
TipTap	-	Editor de texto rico do Estúdio de Escrita
fetch nativo	-	Chamadas HTTP para a API

4.4.2 *Back-end*

A API REST segue o nível 2 do *Richardson Maturity Model*: recursos com nomes claros, verbos HTTP corretos e respostas padronizadas com códigos de status HTTP.

Tabela 7 – Tecnologias de *Back-end*

Tecnologia	**Versão**	**Finalidade**
Java e Spring Boot	21 e 4	API REST, injeção de dependências e validação
Spring Security e JWT	-	Autenticação e proteção das rotas
Spring Data JPA e Flyway	-	Acesso ao banco e versionamento das tabelas
Spring Mail	-	Envio de códigos, convites e alertas por e-mail
Python	-	Serviço de automação que gera os resumos
Swagger (springdoc)	-	Documentação automática da API (RNF08)
JUnit 5 e Mockito	5	Testes unitários (RNF07)

4.4.3 Banco de Dados

O banco é modelado e normalizado até a Terceira Forma Normal (3FN), conforme o RNF06, para evitar dados repetidos e garantir a integridade das ligações entre as tabelas. Cada entidade usa um identificador numérico sequencial (*Long*) como chave primária. Os conteúdos (ideias e projetos, roteiros, materiais, cursos, módulos e aulas) têm também um UUID, que serve como identificador público para as automações (RN05).

Tabela 8 – Tecnologia de Banco de Dados

Tecnologia	**Finalidade**
MySQL 8 (Amazon RDS)	Banco de dados relacional principal, normalizado até a 3FN

4.4.4 Infraestrutura

A infraestrutura reúne os serviços de nuvem e as ferramentas que fazem o sistema funcionar e ser publicado.

4.4.4.1 Amazon Web Services (AWS)

O sistema roda na AWS, dentro de uma VPC. A Tabela 9 mostra os serviços usados e para que cada um serve.

Tabela 9 – Serviços AWS utilizados

Serviço	**Finalidade**
Amazon EKS	Orquestração dos contêineres (Kubernetes)
Amazon ECR	Registro das imagens dos contêineres
Elastic Load Balancer	Entrada do acesso dos usuários, que entrega o tráfego ao *front-end*
Amazon RDS (MySQL)	Banco de dados relacional gerenciado, acessível apenas de dentro da VPC
Amazon S3 e S3 Glacier	Materiais dos projetos e arquivamento dos materiais de cursos publicados há mais de 30 dias (RN04)
Amazon CloudWatch Logs	*Logs* dos contêineres (RNF10)

4.4.4.2 Docker e Contêineres

O sistema roda em três contêineres Docker: o *front-end* (Nginx servindo os arquivos estáticos do Next.js), o *back-end* (Spring Boot) e o serviço de automação em Python. As imagens ficam no Amazon ECR e o Amazon EKS orquestra a execução (RNF09). Os contêineres deixam o ambiente de execução igual entre o desenvolvimento e a produção.

4.4.4.3 Integração Contínua (CI)

O GitHub Actions executa os testes automaticamente a cada envio de código. Os relatórios de cobertura ficam publicados no ambiente de demonstração.

4.5 TESTES E MANUTENIBILIDADE

Os testes e a análise de código ajudam a garantir que o sistema funcione como o esperado e continue fácil de manter.

4.5.1 Plano de Testes

O plano de testes segue uma abordagem em pirâmide. Na base ficam os testes unitários, que são mais baratos de manter, e acima deles os testes de integração e de componentes. O *back-end* tem testes unitários para os casos de uso e serviços, escritos com JUnit 5 e Mockito. O *front-end* tem testes de componentes, da *store* e das funções de acesso à API, escritos com Jest e Testing Library. Os cenários de teste (CT01 a CT17) estão ligados às histórias de usuário na Tabela 5.

4.5.2 Análise Estática

O SonarQube analisa o código a cada entrega e aplica um indicador de qualidade (*quality gate*) ao *back-end*. Os testes rodam automaticamente no GitHub Actions.

4.5.3 Cobertura e Resultados dos Testes

A cobertura dos testes do *back-end* é medida pelo JaCoCo. Na versão final, a cobertura de linhas é superior a 80%, acima da meta de 70% definida no RNF07. A Tabela 10 mostra a cobertura por pacote.

Tabela 10 – Cobertura de testes por pacote (JaCoCo)

Pacote	**Cobertura de linhas**
controller	[PREENCHER]%
usecase	[PREENCHER]%
service	[PREENCHER]%
config	[PREENCHER]%
entity	[PREENCHER]%
Total	**[PREENCHER]%**

O número final de testes e o resultado da execução estão no relatório de testes da entrega: [PREENCHER: número de testes e resultado].

4.6 SEGURANÇA, PRIVACIDADE E LEGISLAÇÃO

A segurança dos dados dos usuários é parte do projeto desde o início. Esta seção mostra as medidas técnicas e como o sistema atende à lei de proteção de dados.

4.6.1 Critérios de Segurança e Privacidade

O sistema tem segurança em várias camadas:

Autenticação e Autorização: o acesso é controlado por funções (RBAC) em cada equipe, com três papéis: Administrador, Criador e Revisor (RF02). O *back-end* confere o papel em toda requisição. Se alguém tentar uma ação sem permissão, a API responde com o código 403. Os *tokens* JWT são assinados e valem 24 horas (RNF03).

Proteção de Dados: as senhas são guardadas somente como *hash* gerado pelo algoritmo BCrypt (RNF01) e nunca aparecem nas respostas da API.

Códigos e Convites: os códigos enviados por e-mail valem 10 minutos e aceitam até 5 tentativas. Os convites valem 7 dias e só podem ser aceitos pela conta com o mesmo e-mail.

Rede: o banco de dados só pode ser acessado de dentro da VPC, e o serviço de automação não fica exposto à internet.

4.6.2 Observância à Legislação

O sistema é desenvolvido de acordo com a Lei Geral de Proteção de Dados Pessoais (LGPD, Lei nº 13.709/2018), que regula o tratamento de dados pessoais no Brasil. As principais medidas são: coleta mínima de dados (apenas nome e e-mail no

cadastro), senhas protegidas com BCrypt e possibilidade de o próprio usuário excluir a conta. Os dados pessoais não são compartilhados com terceiros sem o consentimento do titular.

4.7 MODELO DE BANCO DE DADOS

O modelo de banco de dados organiza as informações do sistema. Esta seção apresenta o MER, o DER, o diagrama de classes e o dicionário de dados.

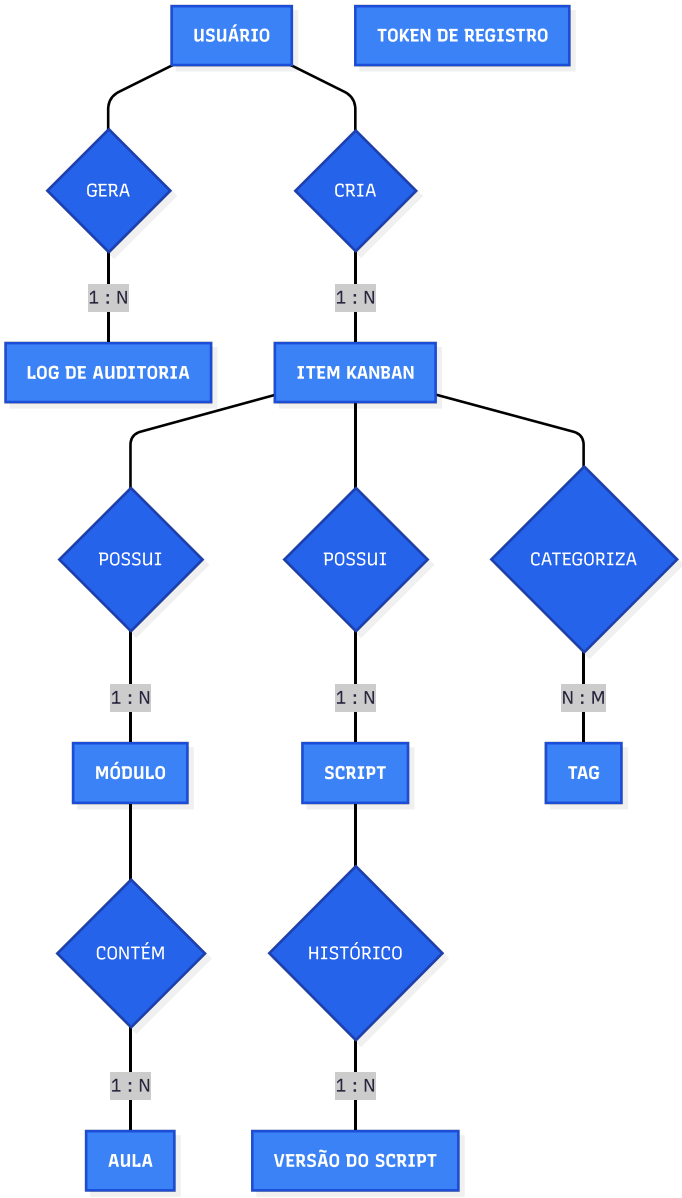
4.7.1 Modelo Entidade-Relacionamento (MER)

O modelo é construído em torno das entidades principais do sistema: Usuário, Equipe, Membro da equipe (que guarda o papel de cada pessoa em cada equipe), Convite, Etiqueta, Coluna, Ideia e Projeto (*card*), Tarefa, Material, Roteiro e suas versões, Revisão, Curso, Módulo e Aula. Cada entidade tem um identificador numérico sequencial como chave primária, e os conteúdos têm também um UUID (RN05).

4.7.2 Figura do Modelo Entidade-Relacionamento

A figura a seguir apresenta o MER do Creator Workbench.

Figura 7 – Modelo Entidade-Relacionamento - Creator Workbench

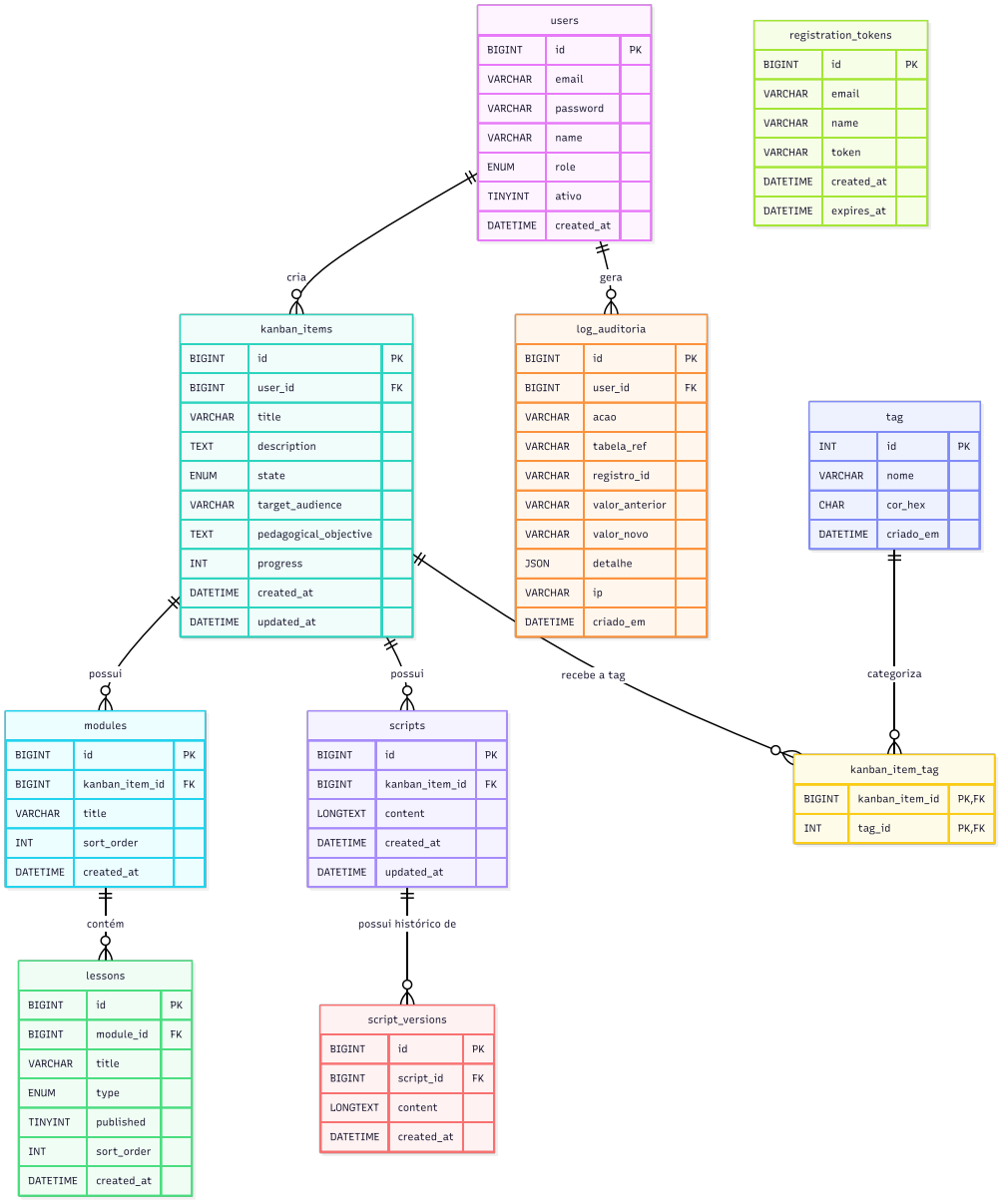


Fonte: Produzido pelos autores.

4.7.3 Diagrama Entidade-Relacionamento (DER)

O DER mostra como o modelo conceitual virou tabelas no banco de dados, com as colunas e as ligações entre elas.

Figura 8 – Diagrama Entidade-Relacionamento - Creator Workbench



Fonte: Produzido pelos autores.

4.7.4 Diagrama de Classes

O diagrama de classes mostra como as entidades do sistema são representadas no código do *back-end* (Figura 9). São 22 classes e 9 enumerações, agrupadas por módulo: usuários e equipes; quadro, ideias e projetos; materiais e roteiro; cursos; e códigos de verificação.

Cada classe mostra seus atributos e métodos, com a visibilidade de cada um. As ligações com losango preenchido indicam composição, em que a parte não existe sem o todo: uma tarefa, por exemplo, só existe dentro de um projeto. As demais ligações são associações simples, com as multiplicidades em cada ponta. Os

atributos marcados com barra (/), como o progresso e a situação da revisão do projeto, são derivados: o sistema os calcula e eles não são gravados no banco. A classe *LessonMaterial* é uma classe de associação, que liga uma aula a um material.

4.7.5 Dicionário de Dados

O dicionário de dados descreve cada campo das principais tabelas: nome, tipo, restrições e significado. Os nomes e os tipos abaixo devem ser conferidos com as migrações Flyway do projeto.

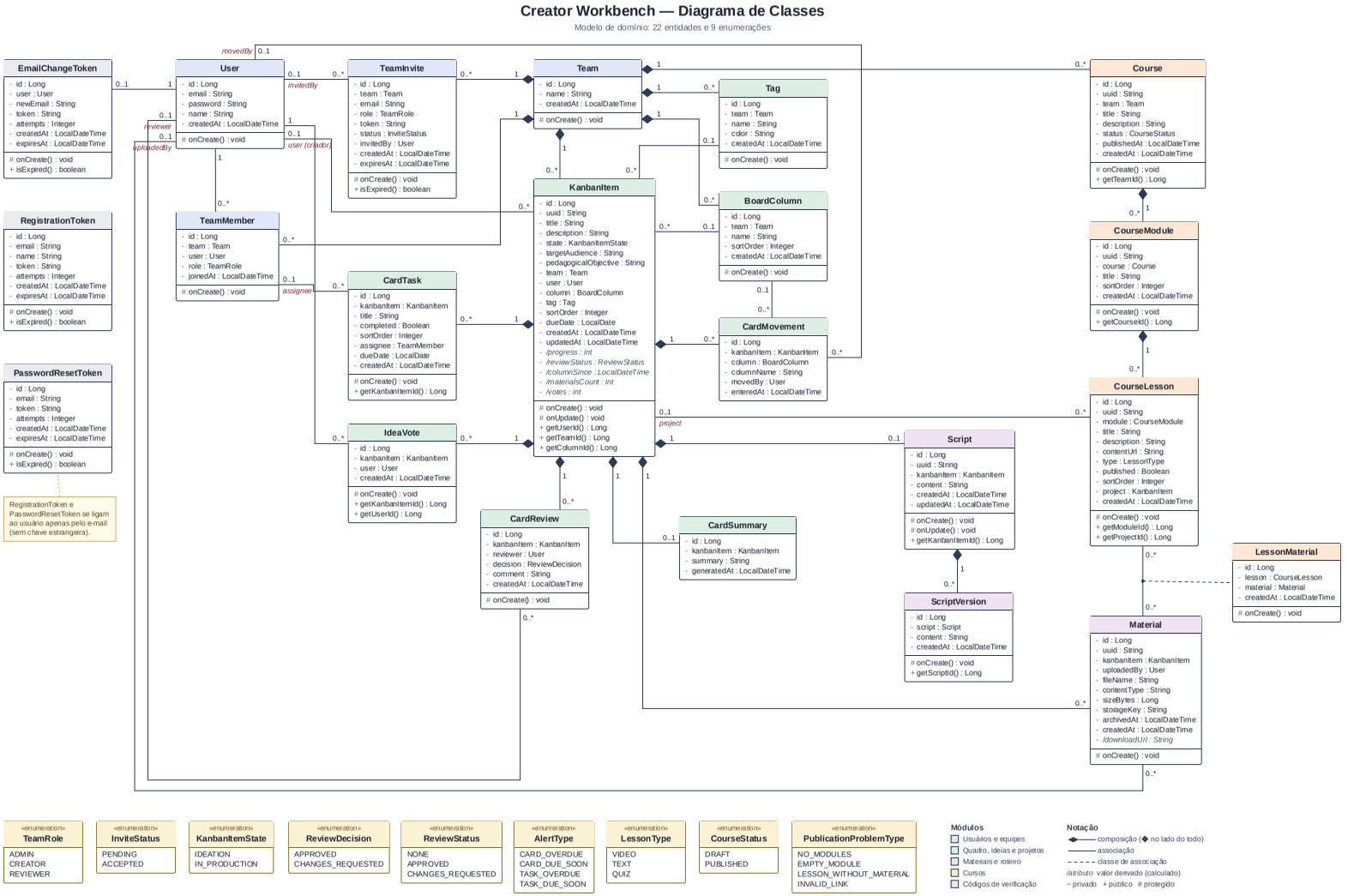
Tabela 11 – Dicionário de Dados: Entidade Usuário

Atributo	**Tipo**	**Restrição**	**Descrição**
id	BIGINT	PK, NOT NULL	Identificador único do usuário
nome	VARCHAR(150)	NOT NULL	Nome completo do usuário
email	VARCHAR(255)	UNIQUE, NOT NULL	E-mail de login e de notificações
senha_hash	VARCHAR(255)	NOT NULL	*Hash* BCrypt da senha
created_at	TIMESTAMP	NOT NULL	Data e hora de criação do registro

Tabela 12 – Dicionário de Dados: Entidade Membro da Equipe

Atributo	**Tipo**	**Restrição**	**Descrição**
id	BIGINT	PK, NOT NULL	Identificador único do membro
usuario_id	BIGINT	FK, NOT NULL	Referência ao usuário
equipe_id	BIGINT	FK, NOT NULL	Referência à equipe
papel	ENUM	NOT NULL	Papel na equipe: ADMINISTRADOR, CRIADOR, REVISOR

Figura 9 – Diagrama de Classes - Creator Workbench



Fonte: Produzido pelos autores.

Tabela 13 – Dicionário de Dados: Entidade Projeto

Atributo	Tipo	Restrição	Descrição
id	BIGINT	PK, NOT NULL	Identificador único do projeto
uuid	CHAR(36)	UNIQUE, NOT NULL	Identificador público para automações (RN05)
titulo	VARCHAR(255)	NOT NULL	Título do conteúdo em produção
publico_alvo	TEXT	NOT NULL	Público-alvo (obrigatório para converter, RN01)
objetivo_pedagogico	TEXT	NOT NULL	Objetivo pedagógico (obrigatório, RN01)
coluna_id	BIGINT	FK, NOT NULL	Coluna do quadro em que o projeto está
prazo	DATE	NULL	Data de entrega do projeto
equipe_id	BIGINT	FK, NOT NULL	Equipe dona do projeto

Fonte: Produzido pelos autores.

4.8 DURAÇÃO / CRONOGRAMA

O cronograma mostra quanto tempo o projeto leva e o que é entregue em cada etapa.

4.8.1 Análise da Duração do Projeto

O projeto é organizado em 8 *Sprints* quinzenais, totalizando aproximadamente 16 semanas de desenvolvimento. A duração foi definida pela complexidade do escopo do MVP, e não pelo calendário do semestre, conforme a orientação metodológica. A Tabela 14 mostra o planejamento inicial, e a Tabela 2, no capítulo de Gestão do Projeto, mostra o que foi de fato entregue em cada *Sprint*.

Tabela 14 – Cronograma de *Sprints* (planejamento inicial)

Sprint	**Duração**	**Entregas Previstas**
Sprint 1	Semanas 1–2	*Setup* da infraestrutura (AWS, Docker, CI), modelagem do banco de dados
Sprint 2	Semanas 3–4	Módulo de autenticação e equipes (RF01, RF02, RNF01, RNF03)
Sprint 3	Semanas 5–6	Módulo de Gestão de Ideias, Caixa de Entrada (RF03, RF04)
Sprint 4	Semanas 7–8	Módulo de Produção, *Kanban* e tarefas (RF05)
Sprint 5	Semanas 9–10	Materiais e Estúdio de Escrita (RF06, RF07)
Sprint 6	Semanas 11–12	Módulo de Automação, resumo e alertas (RF08, RF09)
Sprint 7	Semanas 13–14	Módulo Curricular, Cursos, Módulos, Aulas, exportação (RF10, RF11)
Sprint 8	Semanas 15–16	Testes, ajustes de performance, documentação final

5 VIABILIDADE FINANCEIRA

Este capítulo analisa se o Creator Workbench pode se manter com o dinheiro que arrecada. Primeiro apresenta os custos, depois a receita dos planos de assinatura e, por fim, três cenários de projeção para os primeiros 12 meses.

5.1 CUSTOS

Os custos do projeto são divididos em duas categorias: desenvolvimento (a mão de obra da equipe acadêmica, considerada como custo de oportunidade) e infraestrutura operacional.

A infraestrutura entregue usa o Amazon EKS ligado 24 horas por dia, e por isso o custo é maior que o de um servidor simples. A tabela abaixo mostra a estimativa mensal, com valores de referência sob demanda, na região us-east-1, para 730 horas por mês (**??**). Os valores em reais foram convertidos pela cotação de R$ 5,3420 por dólar, a mesma usada na Tabela 1.

Tabela 15 – Infraestrutura Mensal Estimada

Item	**Dólar**	**Real**
Amazon EKS (plano de controle do *cluster*)	US$ 73,00	R$ 389,97
2 nós EC2 t3.medium do *cluster*	US$ 60,74	R$ 324,47
Elastic Load Balancer (Application Load Balancer)	US$ 22,00	R$ 117,52
Amazon RDS MySQL db.t3.micro com 20 GB	US$ 14,71	R$ 78,58
Discos dos nós (2 x 20 GB EBS gp3)	US$ 3,20	R$ 17,09
Endereços IPv4 públicos (4)	US$ 14,60	R$ 77,99
Amazon S3 e S3 Glacier (cerca de 50 GB)	US$ 2,00	R$ 10,68
Amazon ECR (imagens dos contêineres)	US$ 0,15	R$ 0,80
Amazon CloudWatch Logs (cerca de 5 GB)	US$ 3,00	R$ 16,03
Transferência de dados e envio de e-mails	US$ 4,00	R$ 21,37
Domínio e certificado	-	R$ 10,00
Total mensal	**US$ 197,40**	**R$ 1.064,50**

5.2 RECEITAS

O modelo de monetização proposto é o SaaS (*Software as a Service*), com planos de assinatura mensais. Toda equipe começa no plano Gratuito. O administrador da equipe pode testar uma vez, por 14 dias, o plano Solo, Pro ou Agência, e ao fim do teste a equipe volta ao plano Gratuito. A ativação do plano pago é feita pela administração do sistema, e a integração com um meio de pagamento fica como trabalho futuro.

Tabela 16 – Planos de Assinatura

Plano	Público	Preço Mensal	Funcionalidades
Gratuito	Quem quer conhecer o sistema	R$ 0,00	1 usuário, até 3 projetos ativos, 1 curso, até 100 MB de materiais e sem resumo automático
Solo	Professor autônomo	R$ 49,00	1 usuário, até 10 projetos ativos
Pro	Criador profissional	R$ 99,00	1 usuário, projetos ilimitados, automações
Agência	Pequenas equipes	R$ 249,00	Até 5 usuários, controle de acesso completo (RBAC)

Fonte: Produzido pelos autores.

Como os planos pagos têm preços diferentes, a receita depende de quantos assinantes escolhem cada um. Por isso, em cada cenário a seguir é mostrado o mix de planos usado no cálculo. As premissas da equipe são:

- **Mix de planos pagos:** 65% Solo, 28% Pro e 7% Agência. O valor médio por assinante pagante é de cerca de R$ 77,00 (0,65 $\times$ 49 + 0,28 $\times$ 99 + 0,07 $\times$ 249).
- **Conversão:** 4% dos usuários do plano Gratuito passam a um plano pago, e 20% das equipes que fazem o teste de 14 dias continuam no plano pago.
- **Custo de infraestrutura:** R$ 1.064,50 por mês, como na Tabela 15.

A receita bruta é a soma dos preços de todas as assinaturas pagas, antes de impostos e de taxas de pagamento. O resultado é a receita bruta menos o custo de infraestrutura, e a margem é o resultado dividido pela receita bruta. Com o custo de R$ 1.064,50 e o valor médio de R$ 77,00, a operação se paga a partir de 14 assinantes pagantes. A margem passa de 90% a partir de 139 assinantes pagantes.

5.2.1 Cenário Realista

Projeção para os primeiros 12 meses depois do lançamento, com crescimento moderado, vindo de indicações e de buscas na internet (SEO). Nesse cenário, a maior parte dos assinantes escolhe o plano Solo, que é o mais barato.

Tabela 17 – Mix de planos e receita: Cenário Realista

Mês	Mix de planos e cálculo	Receita Bruta
1 a 3	15 assinantes: 10 Solo (R$ 490,00) + 4 Pro (R$ 396,00) + 1 Agência (R$ 249,00)	R$ 1.135,00
4 a 6	40 assinantes: 26 Solo (R$ 1.274,00) + 11 Pro (R$ 1.089,00) + 3 Agência (R$ 747,00)	R$ 3.110,00
7 a 12	100 assinantes: 65 Solo (R$ 3.185,00) + 28 Pro (R$ 2.772,00) + 7 Agência (R$ 1.743,00)	R$ 7.700,00

Tabela 18 – Projeção Financeira: Cenário Realista

Mês	Assinantes	Receita Bruta	Custo Infra	Resultado	Margem
1 a 3	15	R$ 1.135,00	R$ 1.064,50	R$ 70,50	6,2%
4 a 6	40	R$ 3.110,00	R$ 1.064,50	R$ 2.045,50	65,8%
7 a 12	100	R$ 7.700,00	R$ 1.064,50	R$ 6.635,50	86,2%

Pela conversão de 4%, esses números de assinantes equivalem a cerca de 375, 1.000 e 2.500 usuários no plano Gratuito. Nos primeiros meses o resultado é pequeno, e a operação só cobre o custo com folga a partir do quarto mês.

5.2.2 Cenário Otimista

Adoção mais rápida, com parcerias com produtoras de cursos e lançamento em comunidades de criadores. A meta é chegar a 300 assinantes em 12 meses, com o mesmo mix de planos do cenário realista: 195 Solo (R$ 9.555,00) + 84 Pro (R$ 8.316,00) + 21 Agência (R$ 5.229,00), o que dá uma receita bruta de R$ 23.100,00 por mês. Com o custo de R$ 1.064,50, o resultado é de R$ 22.035,50 e a margem é de 95,4%. Essa margem continua acima de 90% enquanto o custo de infraestrutura ficar abaixo de cerca de R$ 2.310,00 por mês, o que permite um *cluster* maior para esse volume. Pela conversão de 4%, esse cenário equivale a cerca de 7.500 usuários no plano Gratuito.

5.2.3 Cenário Pessimista

Adoção lenta, com 30 assinantes ao final do primeiro ano: 22 Solo (R$ 1.078,00) + 7 Pro (R$ 693,00) + 1 Agência (R$ 249,00), o que dá uma receita bruta de R$ 2.020,00 por mês. Com o custo de R$ 1.064,50, o resultado é de R$ 955,50 e a margem é de 47,3%. Esse valor cobre o custo de infraestrutura e mantém o produto no ar, mas não traz retorno relevante sobre o investimento de desenvolvimento. Nesse cenário, a estratégia para melhorar a situação é divulgar mais o plano Gratuito e o teste de 14 dias, para aumentar o número de usuários e validar o produto.

6 CONSIDERAÇÕES FINAIS

O desenvolvimento do Creator Workbench mostrou que a desorganização enfrentada por criadores de conteúdo educacional digital não é um problema de disciplina pessoal, mas uma falta real no mercado de ferramentas de produtividade. Ferramentas como Trello, Notion e Asana foram feitas para o trabalho de empresas em geral e não têm a estrutura que o domínio educacional precisa. Por isso, o criador tem de adaptar o tempo todo sistemas que nunca foram pensados para a sua rotina.

A proposta do Creator Workbench se mostrou viável tecnicamente e, segundo as projeções do capítulo anterior, também financeiramente. A arquitetura *Client-Server* desacoplada, com *front-end* em Next.js, *back-end* em Java/Spring Boot, serviço de automação em Python e infraestrutura na AWS com contêineres Docker, mantém as camadas independentes e permite que o produto evolua sem prejudicar o que já está funcionando. O modelo de assinatura SaaS, com um plano Gratuito e três planos pagos (Solo, Pro e Agência), cobre o custo de infraestrutura a partir de 14 assinantes pagantes. No cenário realista, a margem chega a cerca de 86% a partir do sétimo mês, e no cenário otimista passa de 90%. Isso confirma que o produto pode crescer sem aumentar os custos na mesma proporção.

A adoção do *Scrum* com *Sprints* quinzenais foi adequada ao projeto. Os ciclos curtos permitiram validar os requisitos aos poucos e corrigir o rumo antes que os desvios ficassem caros. As oito *Sprints* foram concluídas. A principal lição de metodologia foi a importância das revisões de *Sprint* para manter o alinhamento entre os cinco integrantes da equipe, cada um com um papel técnico diferente.

Do ponto de vista acadêmico, o projeto mostrou na prática como as teorias estudadas se ligam às decisões de engenharia: a Andragogia de Knowles, a Arquitetura da Informação de Rosenfeld et al. e os princípios de *ContentOps*. A hierarquia Curso $\rightarrow$ Módulo $\rightarrow$ Aula não é só uma escolha de modelagem de dados. Ela coloca no sistema a ideia andragógica de organizar o conhecimento de forma progressiva.

6.1 DIFICULDADES, ESCOLHAS E DESCARTES

Esta seção reúne o que foi difícil durante o projeto, as escolhas técnicas feitas e o que ficou de fora do MVP.

Dificuldades encontradas: a principal dificuldade técnica foi a integração entre o serviço de automação em Python (RF08) e a API principal em Java/Spring Boot, que exigiu definir com clareza o que cada lado envia e recebe, quanto tempo espera e o que acontece quando o serviço falha. Outra dificuldade foi montar a infraestrutura na AWS, com contêineres, banco de dados, armazenamento e *logs* funcionando juntos.

No lado da organização, coordenar cinco papéis diferentes em um projeto acadêmico com prazos fixos exigiu disciplina no uso das cerimônias do *Scrum*, principalmente das revisões de *Sprint*.

Escolhas justificadas: o Java/Spring Boot foi escolhido para o *back-end*, em vez de alternativas como o Node.js, porque o ecossistema Spring é maduro para criar APIs REST seguras e a equipe já o conhecia. O MySQL foi escolhido em vez de um banco NoSQL porque o domínio é muito relacional: a hierarquia Curso → Módulo → Aula, com suas regras de integridade (RN03), se encaixa bem no modelo relacional. Os contêineres Docker foram escolhidos para deixar o ambiente igual no desenvolvimento e na produção. O serviço de automação ficou separado, em Python, para que o resumo possa mudar no futuro sem mexer na API principal.

Descartes: no levantamento de requisitos, algumas funcionalidades foram consideradas e deixadas de fora do MVP: (1) a integração direta, por API, com plataformas de cursos externas (Hotmart, Udemy), por causa das diferenças entre as APIs de terceiros e do prazo disponível, substituída pela exportação em JSON/CSV (RF11); (2) o módulo de *analytics* do desempenho dos cursos publicados, que dependeria dessas plataformas externas; (3) o aplicativo *mobile*, deixado para depois porque a arquitetura desacoplada já permite criá-lo no futuro; (4) a integração com um meio de pagamento, que ficou de fora porque a ativação dos planos pagos é feita pela administração do sistema nesta versão. Esses itens aparecem na próxima seção como evoluções.

6.2 TRABALHOS FUTUROS

Com o MVP entregue, a equipe identificou evoluções que podem tornar o Creator Workbench mais completo. A Tabela 19 lista essas evoluções com a prioridade sugerida para cada uma.

Tabela 19 – Trabalhos futuros e prioridade

Evolução	**Prioridade**	**Motivo**
Integração com um meio de pagamento para a cobrança dos planos	Alta	Hoje a ativação dos planos pagos é feita pela administração do sistema, e a cobrança automática permite crescer
Integração direta com APIs de plataformas de cursos (Hotmart, Eduzz)	Média	Elimina a exportação manual em JSON/CSV
Módulo de *analytics* de desempenho dos cursos	Média	Mostra ao criador como os cursos publicados estão indo
Aplicativo *mobile* nativo	Média	A arquitetura desacoplada já permite criar novos clientes sem mudar a API
Inteligência artificial generativa para sugerir roteiros	Baixa	Combina com o curso de ADS, mas depende de custo e de estudo de viabilidade

REFERÊNCIAS

Asana, Inc. **Asana: preços e planos**. 2026. Acesso em: 3 nov. 2026. Disponível em: <https://asana.com/pricing>. Citado 2 vezes nas páginas 19 e 20.

Atlassian. **Trello: preços e planos**. 2026. Acesso em: 3 nov. 2026. Disponível em: <https://trello.com/pricing>. Citado 2 vezes nas páginas 18 e 20.

Banco Central do Brasil. **Cotações e boletins: dólar PTAX**. 2026. Acesso em: 3 nov. 2026. Disponível em: <https://www.bcb.gov.br/estabilidadefinanceira/historicocotacoes>. Citado na página 20.

BOIKO, B. **Content Management Bible**. [S.l.]: Wiley, 2004. Citado na página 21.

BUNJAK, A.; ČERNE, M.; POPOVIč, A. Absorbed in technology but digitally overloaded: interplay effects on gig workers' burnout and creativity. **Information & Management**, v. 58, n. 8, p. 103533, 2021. Citado na página 17.

DEMENY, M. **Some Thoughts on the Gartner Digital Experience Platforms (DXP) Magic Quadrant for 2025**. 2025. CMS Critic. Acesso em: 3 nov. 2026. Disponível em: <https://cmscritic.com/some-thoughts-on-the-gartner-digital-experience-platforms-dxp-magic-quadrant-for-2025>. Citado na página 22.

DRUCKER, P. F. Knowledge-worker productivity: the biggest challenge. **California Management Review**, v. 41, n. 2, p. 79–94, 1999. Citado 2 vezes nas páginas 16 e 21.

Goldman Sachs. **The creator economy could approach half-a-trillion dollars by 2027**. 2023. Acesso em: nov. 2026. Disponível em: <https://www.goldmansachs.com/insights/articles/the-creator-economy-could-approach-half-a-trillion-dollars-by-2027>. Acesso em: nov. 2026. Citado na página 17.

Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira. **Censo da Educação Superior 2024: notas estatísticas**. Brasília, DF: Inep, 2025. Disponível em: <https://download.inep.gov.br/publicacoes/institucionais/estatisticas_e_indicadores/notas_estatisticas_censo_escolar_da_educacao_superior_2024.pdf>. Acesso em: 4 nov. 2026. Citado na página 15.

KNOWLES, M. **The Modern Practice of Adult Education: Pedagogy vs. Andragogy**. [S.l.]: Association Press, 1970. Citado na página 22.

KNOWLES, M.; HOLTON, E.; SWANSON, R. **The Adult Learner**. [S.l.]: Routledge, 2015. Citado na página 22.

MCKNIGHT, C. **Don't Avoid Content Ops — Use This Helpful Framework Now**. 2023. Content Marketing Institute. Acesso em: 3 nov. 2026. Disponível em: <https://contentmarketinginstitute.com/articles/scale-content-operations/>. Citado na página 21.

Notion Labs. **Notion: preços e planos**. 2026. Acesso em: 3 nov. 2026. Disponível em: <https://www.notion.com/pricing>. Citado 2 vezes nas páginas 18 e 20.

ROSENFELD, L.; MORVILLE, P.; ARANGO, J. **Information Architecture: For the Web and Beyond**. 4. ed. [S.l.]: O'Reilly Media, 2015. Citado 2 vezes nas páginas 21 e 23.

SCHWABER, K.; SUTHERLAND, J. **O Guia do Scrum: o guia definitivo para o Scrum — as regras do jogo**. Scrum.org, 2020. Disponível em: <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-PortugueseBR-3.0.pdf>. Acesso em: abr. 2026. Citado na página 25.

Apêndices

APÊNDICE A – TELAS DA APLICAÇÃO – CREATOR WORKBENCH

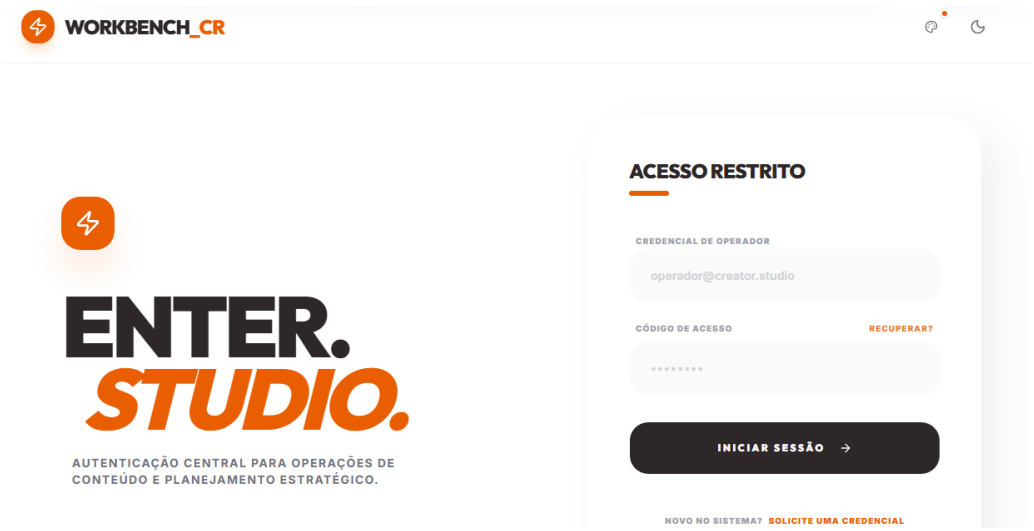


Figura 10 – Tela de Autenticação – Acesso restrito com campos de credencial e código de acesso

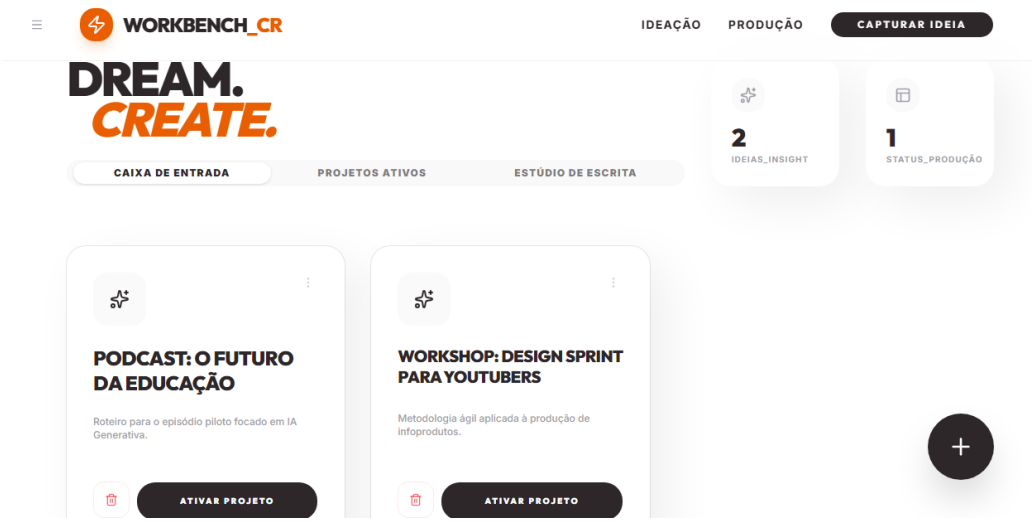


Figura 11 – *Dashboard* Principal – Caixa de Entrada com *cards* de ideias e contadores de *Ideias_Insight* e Status_Produção

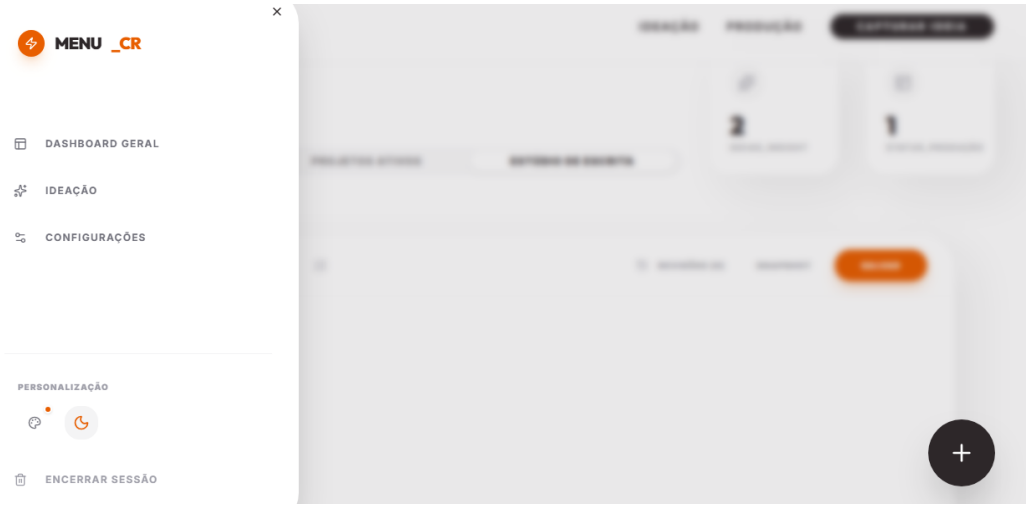


Figura 12 – Menu Lateral – Navegação entre Dashboard Geral, Ideação e Configurações, com opções de personalização e encerramento de sessão

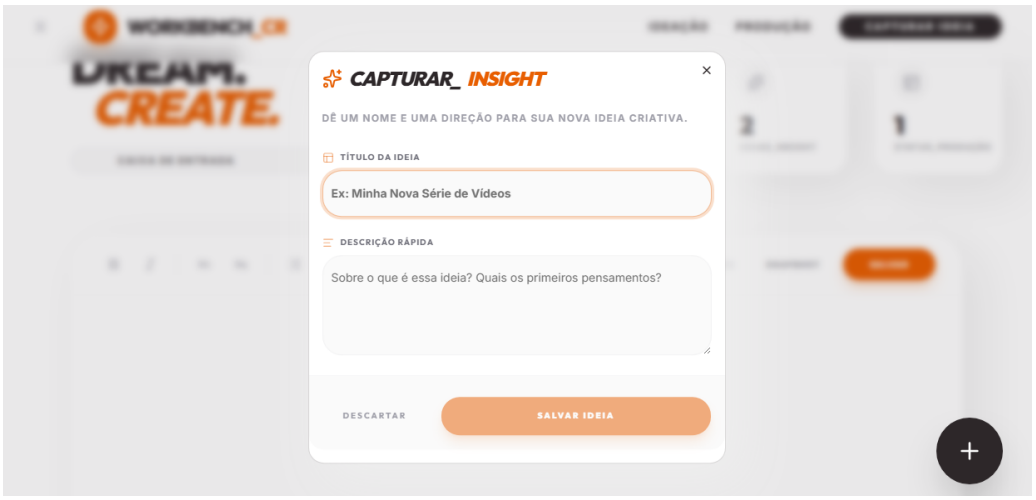


Figura 13 – *Modal* Capturar *Insight* – Formulário de cadastro rápido de ideia com campos de título e descrição

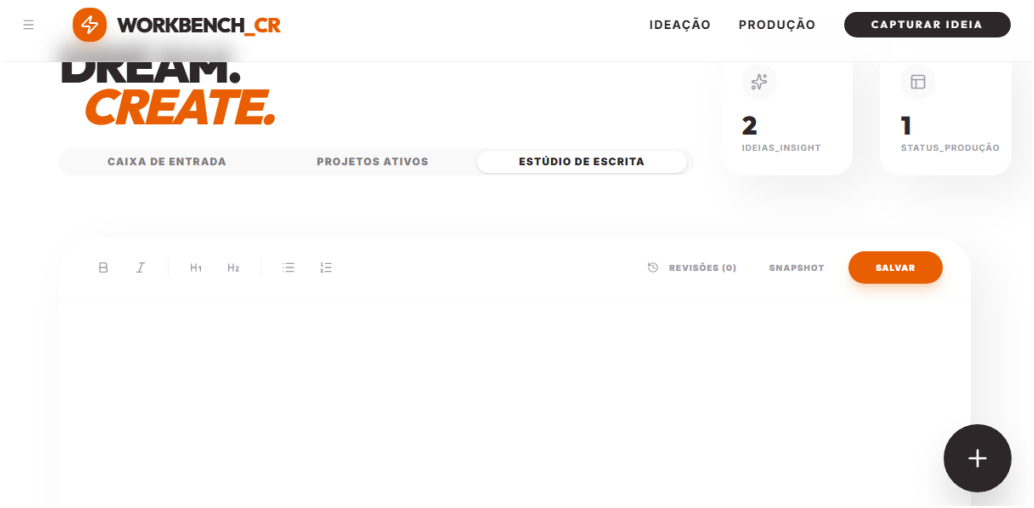


Figura 14 – Estúdio de Escrita – Editor de texto rico com suporte a formatação, revisões e snapshot de versões



Figura 15 – Projetos Ativos – *Pipeline_Ativo* com projeto em produção e painel de Estrutura Curricular com módulos e aulas planejadas